Team Notebook

CCL - dulces tea

1 C++_utils 1
1.1 c++ template 1
1.2 c++ cheat sheet 2
1.3 fraction 3
2 Data_structures 3
2.1 C++ STL / bitset 3
2.2 C++ STL / policy based ordered set 3
2.3 C++ STL / set & multiset 4
2.4 fenwick tree 4
2.5 fenwick tree 2d 5
2.6 segment tree 5
2.7 segment tree lazy 6
2.8 sparse table 7
2.9 unionfind 8
3 Strings 8
3.1 aho corasick 8
3.2 kmp 8
3.3 rolling hashing 9
3.4 shortest repeating cycle 10
3.5 suffix array 10
3.6 trie 10
4 Divide&conquer 10
4.1 mergesort 10
5 Mathematics 11
5.1 binary modular exponentiation 11
5.2 chinese remainder theorem 11
5.3 euclidean algorithm 11
5.4 integer root square 12
5.5 modular binomial coefficient 12
5.6 modular fibonacci 13
5.7 modular multinomial coefficient 13
5.8 primality test 13
5.9 prime factorization 14
5.10 sieve of eratosthenes 14
6 Dynamic_programming 14
6.1 divide&conqueroptimization 14
6.2 lis 15
6.3 tsp 15
7 Search 15
7.1 binarysearch 15
7.2 ternarysearch 16
8 Graphs 16
8.1 articulation points, cut edges, biconnected components 16
8.2 bellman ford 17
8.3 diameter of tree 17
8.4 dijkstra 18
8.5 dinic 18
8.6 floyd warshall 18
8.7 level ancestor 18
8.8 lowest common ancestor 19
8.9 minimum spanning tree 20
8.10 strongly connected components 20
8.11 toposort 21
9 Geometry 21
9.1 2D / point2d 21
9.2 2D / common 2d algorithms 21
9.3 closest pair of points 23
9.4 convex hull 24
9.5 greens theorem 24
9.6 smallest enclosing circle 24
9.7 trigonometry 24
10 Apuntes Javi 25
10.1 *template 25
10.2 conv binpow 25
10.3 directed mst 25
10.4 fft de dudosa procedencia 26
10.5 ntt 26
10.6 segment tree 26
10.7 segment tree lazy 26

1 C++_utils

1.1 c++ template

```cpp
#pragma GCC optimize("Ofast")
#include <bits/stdc++.h>
using namespace std;
// defines
#define rep(i,a,b) for(int i = a; i < b; ++i) // [a, b), inclusive-exclusive
#define invrep(i,b,a) for(int i = b; i >= a; --i) // [b, a], inclusive-inclusive
#define umap unordered_map
#define uset unordered_set
#define ff first
#define ss second
#define pb push_back
#define eb emplace_back
// typedefs
typedef vector<int> vi;
typedef pair<int,int> ii;
typedef unsigned long long int ull;
typedef long long int ll;
// ------------------------------
signed main() { // signed allows using #define int long long
    ios::sync_with_stdio(false); cin.tie(0);
    return 0;
}
```

1.2 c++ cheat sheet

```cpp
/* ================================ */
/* Input/Output with C++: cin & cout */
/* ================================ */
// reading many lines of unknown length
string line;
while(getline(cin, line)) {}
// reading tokens from a line
string token;
stringstream ss(line);
while (ss >> token) { /* do something with token */}
// printing floating with fixed precision
cout << setprecision(6) << fixed;
cout << 12312.12312355;

/* ================================ */
/* CONVERTING FROM STRING TO NUMBERS */
/* ================================ */
// 1) stringstream
string s = "12345";
stringstream ss(s);
int x;
ss >> x; // x = 12345
ss << "12345678910";
long long y;
ss >> y; // y = 12345678910
// 2) stoi, stoll
string str_dec = "2001, A Space Odyssey";
string str_hex = "40c3";
string str_bin = "-10010110001";
string str_auto = "0x7f";
int sz;
int i_dec = stoi(str_dec,&sz);
int i_hex = stoi(str_hex,0,16);
int i_bin = stoi(str_bin,0,2);
int i_auto = stoi(str_auto,0,0);
cout << str_dec << ": " << i_dec << " and [" <<
str_dec.substr(sz) << "]\n";
cout << str_hex << ": " << i_hex << '\n';
cout << str_bin << ": " << i_bin << '\n';
cout << str_auto << ": " << i_auto << '\n';
// 2001, A Space Odyssey: 2001 and [, A Space Odyssey]
// 40c3:  16579
// -10010110001: -1201
// 0x7f: 127
string str = "8246821 0xffff 020";
int sz = 0;
while (!str.empty()) {
    long long ll = stoll(str,&sz,0);
    cout << str.substr(0,sz) << " interpreted as " << ll <<
'\n';
    str = str.substr(sz);
}
// 8246821 interpreted as 8246821
//  0xffff interpreted as 65535
//  020 interpreted as 16

/* ========================== */
/* C STRING UTILITY FUNCTIONS */
/* ========================== */
int strcmp ( const char * str1, const char * str2 ); //
(-1,0,1)
int memcmp ( const void * ptr1, const void * ptr2, size_t
num ); // (-1,0,1)
void * memcpy ( void * destination, const void * source,
size_t num );

/* ============================ */
/* C++ STRING UTILITY FUNCTIONS */
/* ============================ */
// split a string by a single char delimiter
void split(const string &s, char delim, vector<string>
&elems) {
    stringstream ss(s);
    string item;
    while (getline(ss, item, delim))
        elems.push_back(item);
}
// find index of string or char within string
string str = "random";
size_t pos = str.find("ra");
size_t pos = str.find('m');
if (pos == string::npos) // not found
// substrings
string subs = str.substr(pos, length);
string subs = str.substr(pos); // default: to the end of the
string
// std::string from cstring's substring
const char* s = "bla1 bla2";
int offset = 5, len = 4;
string subs(s + offset, len); // bla2

// -------------------------
// string comparisons
string str1("green apple");
string str2("red apple");
if (str1.compare(str2) != 0)
    cout << str1 << " is not " << str2 << '\n';
if (str1.compare(6,5,"apple") == 0)
    cout << "still, " << str1 << " is an apple\n";
if (str2.compare(str2.size()-5,5,"apple") == 0)
    cout << "and " << str2 << " is also an apple\n";
if (str1.compare(6,5,str2,4,5) == 0)
    cout << "therefore, both are apples\n";

/* ==================== */
/*  OPERATOR OVERLOADING */
/* ==================== */
//-------------------------
// method #1: inside struct
struct Point {
    int x, y;
    bool operator<(const Point& p) const {
        if (x != p.x) return x < p.x;
        return y < p.y;
    }
};
//-------------------------
// method #2: outside struct
struct Point {int x, y; };
bool operator<(const Point& a, const Point& b) {
    if (a.x != b.x) return a.x < b.x;
    return a.y < b.y;
}
// Note: if you overload the < operator for a custom struct,
// then you can use that struct with any library function
// or data structure that requires the < operator
// Examples:
priority_queue<Point> pq;
vector<Point> pts;
sort(pts.begin(), pts.end());
lower_bound(pts.begin(), pts.end(), {1,2});
upper_bound(pts.begin(), pts.end(), {1,2});
set<Point> pt_set;
map<Point, int> pt_map;

/* ========================================================== */
/* Lambda Functions                                           */
/* Syntax: [capture](parameters) -> return_type { body }; */
/* ========================================================== */
// ------------------------------------------------------------
// 1. Capture Clauses (How lambda sees external variables)
// ------------------------------------------------------------
// []        : Captures nothing. (Required for C++20
decltype tricks)
// [&]       : Captures all variables by reference.
(Fastest, go-to for CP)
// [=]       : Captures all variables by value/copy.
(Slower, uses more memory)
// [x, &y]   : Captures 'x' by value, 'y' by reference.

// ------------------------------------------------------------
// 2. Standard Usage: Sorting based on external arrays
// ------------------------------------------------------------
// Very common in CP: Sorting an array of indices based on
values in another array.
vector<int> H = {50, 20, 40, 10, 30};
vector<int> idxs(5);
iota(idxs.begin(), idxs.end(), 0); // Fills idxs with 0, 1,
2, 3, 4 (requires <numeric>)
// We use [&] so the lambda can read the vector 'H' without
copying it.
auto cmp_idx = [&](int i, int j) { return H[i] < H[j]; };
sort(idxs.begin(), idxs.end(), cmp_idx); // idxs is now: 3,
```

```
1, 4, 2, 0

// ------------------------------------------------------------
// 3. Lambdas with Sets/Maps/Priority Queues (C++11 / C++14 / C++17)
// ------------------------------------------------------------
// Before C++20, lambdas had no default constructor. To use a custom
// comparator in a set/map/pq, you MUST pass the lambda type using
// decltype AND pass the lambda instance to the container's constructor.

// Note: Use [] (empty capture) for container comparators.
auto cmp_desc = [](int a, int b) { return a > b; };
// 1. Pass type via decltype(cmp_desc)
// 2. Pass instance via constructor (cmp_desc)
set<int, decltype(cmp_desc)> my_set(cmp_desc);
my_set.insert(1);
my_set.insert(5); // Set contents: 5, 1
// Same rule applies for priority_queue:
priority_queue<int, vector<int>, decltype(cmp_desc)>
pq(cmp_desc);

// ------------------------------------------------------------
// 4. Generic Lambdas (C++14)
// ------------------------------------------------------------
// You can use 'auto' in parameters. Great for writing a single comparator
// that works for vector<int>, vector<long long>, or vector<pair<int, int>>.
auto cmp_generic = [](const auto& a, const auto& b) { return a > b; };

/* =============== */
/* RANDOM INTEGERS */
/* =============== */
srand(time(NULL));
int x = rand() % 100; // 0-99
int randBetween(int a, int b) { // a-b
    return a + (rand() % (1 + b - a));
}

/* ============== */
/* Bitwise Tricks */
/* ============== */
// amount of one-bits in number
int __builtin_popcount(int x);
int __builtin_popcountl(long x);
int __builtin_popcountll(long long x);
// amount of leading zeros in number
int __builtin_clz(int x);
int __builtin_clzl(long x);
int __builtin_clzll(ll x);
// binary length of non-negative number
int bitlen(int x) { return sizeof(x) * 8 - __builtin_clz(x); }
int bitlen(ll x) { return sizeof(x) * 8 - __builtin_clzll(x); }
// index of most significant bit
int log2(int x) { return sizeof(x) * 8 - __builtin_clz(x) - 1; }
int log2(ll x) { return sizeof(x) * 8 - __builtin_clzll(x) - 1; }
// reverse the bits of an integer
int reverse_bits(int x) {
    int v = 0;
    while (x) v <<= 1, v |= x&1, x >>= 1;
    return v;
}
// get string binary representation of an integer
string bitstring(int x) {
    int len = sizeof(x) * 8 - __builtin_clz(x);
    if (len == 0) return "0";
    char buff[len+1]; buff[len] = '\0';
    for (int i = len-1; i >= 0; --i, x >>= 1)
        buff[i] = (char)('0' + (x&1));
    return string(buff);
}

/* ================== */
/* Hexadecimal Tricks */
/* ================== */
// get string hex representation of an integer
std::stringstream stream;
stream << std::hex << your_int;
std::string result(stream.str());

/* ================ */
/* CLIMITS CONSTANTS */
/* ================ */
INT_MIN INT_MAX UINT_MAX LONG_MIN LONG_MAX ULONG_MAX
LLONG_MIN LLONG_MAX ULLONG_MAX
```

1.3 fraction

```
template<typename T> struct Fraction {
    T x, y; // x / y
    Fraction() : x(0), y(1) {}
    Fraction(T x_, T y_) : x(x_), y(y_) { if (y < 0) x = -x, y = -y; simplify(); }
    bool operator<(const Fraction& o) const { return x * o.y < y * o.x; }
    bool operator==(const Fraction& o) const { return x * o.y == y * o.x; }
    bool operator<=(const Fraction& o) const { return x * o.y <= y * o.x;}
    Fraction operator+(const Fraction& o) const {
        Fraction ans = {x * o.y + y * o.x, y * o.y};
ans.simplify(); return ans;
    }
    Fraction operator-(const Fraction& o) const {
        Fraction ans = {x * o.y - y * o.x, y * o.y};
ans.simplify(); return ans;
    }
    Fraction operator*(const Fraction& o) const {
        Fraction ans = {x * o.x, y * o.y}; ans.simplify();
return ans;
    }
    void simplify() {
        T g = __gcd(x, y); x /= g; y /= g;
    }
};
```

2 Data_structures

2.1 C++ STL / bitset

```
bitset<4> foo; // 0000
foo.size(); // 4
foo.set(); // 1111
foo.set(1,0); // 1011
foo.test(1); // false
foo.set(1); // 1111
foo.test(1); // true
```

2.2 C++ STL / policy based ordered set

```
// references:
// https://www.geeksforgeeks.org/ordered-set-gnu-c-pbds/
// https://www.geeksforgeeks.org/policy-based-data-structures-g/
// https://codeforces.com/blog/entry/11080
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> ordered_set;
int main() {
    ordered_set o_set;
    o_set.insert(5);
    o_set.insert(1);
    o_set.insert(2);
    // Finding the second smallest element
    // in the set using * because
    //  find_by_order returns an iterator
    cout << *(o_set.find_by_order(1)) << '\n';
    // Finding the number of elements
    // strictly less than k=4
    cout << o_set.order_of_key(4) << '\n';
    // Finding the count of elements less
    // than or equal to 4 i.e. stricly less
```

```
    // than 5 if integers are present
    cout << o_set.order_of_key(5) << '\n';
    // Deleting 2 from the set if it exists
    if (o_set.find(2) != o_set.end())
        o_set.erase(o_set.find(2));
    // Now after deleting 2 from the set
    // Finding the second smallest element in the set
    cout << *(o_set.find_by_order(1)) << '\n';
    // Finding the number of
    // elements strictly less than k=4
    cout << o_set.order_of_key(4) << '\n';
    return 0;
}
```

2.3 C++ STL / set & multiset

```
// references:
// http://www.cplusplus.com/reference/set/set/
// http://www.cplusplus.com/reference/set/multiset/
#define rep(i,a,b) for(int i=a; i<=b; i++)

//================
// INITIALIZATION
//================
// set
set<int> s{1, 2, 3, 4, 4, 5, 5, 5, 2, 2, 2};
for (int x : s) cout << x; // 12345
// multiset
multiset<int> ms{1, 2, 3, 4, 4, 5, 5, 5, 2, 2, 2};
for (int x : s) cout << x; // 12222344555

//========
// INSERT
//========
// set
set<int> s;
rep(i,1,5) s.insert(i*10);    // 10 20 30 40 50
auto ret = s.insert(20);  // no new element inserted
auto it = ret.first;
if (it.second) cout << "20 inserted for the first time\n";
else cout << "20 already in set\n";
int myints[]= {5,10,15};     // 10 already in set, not
inserted
s.insert(myints,myints+3);
cout << "s contains:";
for (int x : s) cout << ' ' << x;
cout << '\n'; // 5 10 15 20 30 40 50
// multiset
// ** same as set, but allows duplicates, so insert returns
an iterator
//    not a pair

//=======
// ERASE
//=======
```

```
// -- set
set<int> s;
rep(i,1,9) s.insert(i*10);  // 10 20 30 40 50 60 70 80 90
auto it = s.begin();
++it; // "it" points now to 20
s.erase(it); // erase by pointer
s.erase(40); // erase by value
it = s.find(60); // iterator pointing to 60
s.erase(it, s.end()); // erase everything in range [it,
s.end())
// s = 10 30 50
// -- multiset
multiset<int> ms;
ms.insert (40);               // 40
rep(i,1,6) ms.insert(i*10);   // 10 20 30 40 40 50 60
auto it=ms.begin();
it++;                         //    ^
ms.erase(it);                 // 10 30 40 40 50 60
ms.erase(40);                 // 10 30 50 60
it=ms.find(50);
ms.erase(it, ms.end());       // 10 30

//=======
// FIND
//=======
// -- set
set<int> s;
rep(i,1,5) s.insert(i*10);  // 10 20 30 40 50
auto it=s.find(20);
s.erase(it);                  // 10 30 40 50
s.erase(s.find(40));          // 10 30 50
// -- multiset
// ** same as set

//================================
// lower_bound() & upper_bound()
//================================
// -- set
set<int> s;
rep(i,1,9) s.insert(i*10);    // 10 20 30 40 50 60 70 80 90
auto itlow=s.lower_bound(30); //       ^
auto itup=s.upper_bound(60);  //                   ^
s.erase(itlow,itup);          // 10 20 70 80 90
// -- multiset
multiset<int> ms{30, 10, 10, 40, 30, 90}; // 10 10 30 30 40
90
auto itlow = ms.lower_bound(30);          //       ^
auto itup = ms.upper_bound(40);           //
^
ms.erase(itlow,itup);                     // 10 10 90

//======================
// multiset::equal_range
//======================
int myints[] = {77,30,16,2,30,30};
```

```
multiset<int> ms(myints, myints+6);  // 2 16 30 30 30 77
auto ret = ms.equal_range(30);       //      ^        ^
// ret.first -> first 30 (same as ms.lower_bound(30))
// ret.second -> 77 (same as ms.upper_bound(30))
ms.erase(ret.first, ret.second);     // 2 16 77

//=======
// COUNT
//=======
// --- set
set<int> s{3, 6, 9, 12};
rep(i,0,9) {
    cout << i;
    if (s.count(i) > 0) cout << " is an element of s.\n";
    else cout << " is not an element of s.\n";
}
// --- multiset
multiset<int> ms{10,73,12,22,73,73,12};
cout << ms.count(73); // 3

//============================
// SET/MULTISET of Custom Data
//============================
struct CustomData {
    int x; string name;
    CustomData(int x, string n) : x(x), name(n) {}
    // define operator <
    bool operator<(const CustomData& rhs) const {
        return x < rhs.x;
    }
};
set<CustomData> s;
multiset<CustomData> ms;
s.emplace(1, "foo");
s.emplace(2, "bar");
ms.emplace(-12, "bla");
```

2.4 fenwick tree

```
struct FT { // fenwick tree
    vector<int> t;
    FT(int n) { t.assign(n+1, 0); }
    int query(int i) { // sum in range 1 .. i
        int ans = 0;
        for (; i; i -= (i & -i)) ans += t[i];
        return ans;
    }
    int query(int i, int j) { return query(j) -
query(i-1); } // sum in range [i .. j]
    void update(int i, int v) { // increment i'th value by v
(and propagate)
        for (; i < t.size(); i += i & (-i)) t[i] += v;
    }
    void update(int i, int j, int v) { update(i, v);
```

```
update(j + 1, -v); }
};
```

2.5 fenwick tree 2d

```
template<typename T> struct FT2D {
    vector<vector<T>> ft;
    int R, C;
    FT2D(int _R, int _C) : R(_R+1), C(_C+1) {
        ft.assign(R, vector<T>(C, 0));
    }
    void add(int r, int c, T val) {
        for (int i = r; i < R; i += (i&-i))
            for (int j = c; j < C; j += (j&-j))
                ft[i][j] += val;
    }
    // sum[(1, 1), (r, c)]
    T sum(int r, int c) {
        T res = 0;
        for (int i = r; i; i -= (i&-i))
            for (int j = c; j; j -= (j&-j))
                res += ft[i][j];
        return res;
    }
    // sum[(r1, c1), (r2, c2)]
    T sum(int r1, int c1, int r2, int c2) {
        return sum(r2, c2) - sum(r1-1, c2) - sum(r2, c1-1) +
sum(r1-1, c1-1);
    }
    // get value at cell (r,c)
    T get(int r, int c) { return sum(r, c, r, c); }
    // set value to cell (r,c)
    void set(int r, int c, int value) {
        add(r, c, -get(r, c) + value);
    }
};
```

2.6 segment tree

```
//============================
// 1) Segment Tree - ITERATIVE
//============================
// source: https://docs.google.com/document/d/1rcex_saP4
tExbbU62qGUjR3eenxOh-50i9Y45WtHkc4/
/*
Se requiere un struct para el nodo (ej: prodsgn).
Un nodo debe tener tres constructores:
    Aridad 0: Construye el neutro de la operacion
    Aridad 1: Construye un nodo hoja a partir del input
    Aridad 2: Construye un nodo segun sus dos hijos

Construccion del segment tree:
    Hacer un arreglo de nodos (usar ctor de aridad 1).
    ST<miStructNodo> miSegmentTree(arregloDeNodos);
Update:
    miSegmentTree.set_point(indice, miStructNodo(input));
Query:
    miSegmentTree.query(l, r) es inclusivo exclusivo y da un
nodo. Usar la info del nodo para obtener la respuesta.
 */
template<class node> struct ST {
    vector<node> t; int n;
    ST(vector<node> &arr) {
        n = arr.size();
        t.resize(n*2);
        copy(arr.begin(), arr.end(), t.begin() + n);
        for (int i = n-1; i > 0; --i)
            t[i] = node(t[i<<1], t[i<<1|1]);
    }
    // 0-indexed
    void set_point(int p, const node &value) {
        for (t[p += n] = value; p > 1; p >>= 1)
            t[p>>1] = node(t[p], t[p^1]);
    }
    // inclusive exclusive, 0-indexed
    node query(int l, int r) {
        node ansl, ansr;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l&1) ansl = node(ansl, t[l++]);
            if (r&1) ansr = node(t[--r], ansr);
        }
        return node(ansl, ansr);
    }
};

// Range Minimum Query
struct rminq {
    int value;
    rminq() {value = INT_MAX;}
    rminq(int x) {value = x;}
    rminq(const rminq &a, const rminq &b) {
        value = min(a.value, b.value);
    }
};

//============================
// 2) Segment Tree - RECURSIVE
//============================
template<class t> class ST {
    vector<ll> *arr, st; int n;

    void build(int u, int i, int j) {
        if (i == j) {
            st[u] = (*arr)[i];
            return;
        }
        int m = (i+j)/2, l = u*2+1, r = u*2+2;
        build(l, i, m);
        build(r, m+1, j);
        st[u] = t::merge_op(st[l], st[r]);
    }
    ll query(int a, int b, int u, int i, int j) {
        if (j < a or b < i) return t::neutro;
        if (a <= i and j <= b) return st[u];
        int m = (i+j)/2, l = u*2+1, r = u*2+2;
        ll x = query(a, b, l, i, m);
        ll y = query(a, b, r, m+1, j);
        return t::merge_op(x, y);
    }
    void update(int a, ll value, int u, int i, int j) {
        if (j < a or a < i) return;
        if (i == j) st[a] += value;
        else {
            int m = (i+j)/2, l = u*2+1, r = u*2+2;
            update(a, value, l, i, m);
            update(a, value, r, m+1, j);
            st[u] = t::merge_op(st[l], st[r]);
        }
    }

public:
    ST(vector<ll>& v) {
        arr = &v;
        n = v.size();
        st.resize(n*4+5);
        build(0, 0, n-1);
    }
    ll query(int a, int b) {
        return query(a, b, 0, 0, n-1);
    }
    void update(int a, ll value) { // adding value to
position a
        update(a, value, 0, 0, n-1);
    }
};

struct RSQ { // range sum query
    static ll const neutro = 0;
    static ll merge_op(ll x, ll y) { return x + y; }
};
struct RMinQ { // range minimun query
    static ll const neutro = LLONG_MAX;
    static ll merge_op(ll x, ll y) { return min(x, y); }
};
struct RMaxQ { // range maximum query
    static ll const neutro = LLONG_MIN;
    static ll merge_op(ll x, ll y) { return max(x, y); }
};

// usage
int main() {
    vector<int> A = { 18, 17, 13, 19, 15, 11, 20 };
    ST<RSQ> st(A);
    st.update(2, 100);
    st.query(1, 3);
```

```
    return 0;
}
```

2.7 segment tree lazy

```
/*
================================================================================
*/
/* Lazy Segment Tree (based on AtCoder Library's
lazy_segtree, needs C++17)  */
/*
================================================================================
*/
// Source: https://github.com/atcoder/ac-library/blob/
master/atcoder/lazysegtree.hpp
// Docs:   https://github.com/atcoder/ac-library/blob/
master/document_en/lazysegtree.md
//
// Supports, all in O(log n):
//   - apply an update f to every element of a range
//   - query the "sum" (any associative op) of a range
//   - binary search on the tree (max_right / min_left)
//
// ALL RANGES ARE 0-INDEXED AND HALF-OPEN: [l, r)
//
// You must define (see examples below):
//   S                  type stored in each node (often a
struct, e.g. {sum, len})
//   S op(S a, S b)     combine two adjacent nodes (must be
associative)
//   S e()              identity of op: op(e(), x) == op(x,
e()) == x
//   F                  type of an update
//   S mapping(F f, S x)        apply update f to a node x
//   F composition(F f, F g)    the update "first g, then
f" (f is the NEWER one)
//   F id()             the "do nothing" update
//
// Tip: if the update depends on the segment length (e.g.
range add + range sum),
// store the length inside S, since mapping() doesn't
receive the range.
template <class S, auto op, auto e, class F, auto mapping,
auto composition, auto id>
struct LazySegTree {
    LazySegTree(int n) : LazySegTree(vector<S>(n, e())) {}
    LazySegTree(const vector<S>& v) : _n(v.size()) {
        log = 0;
        while ((1 << log) < _n) log++;
        size = 1 << log; // smallest power of 2 >= n
        d.assign(2 * size, e());
        lz.assign(size, id());
        // node k has children 2k and 2k+1; leaves are
size..size+n-1
        for (int i = 0; i < _n; i++) d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--) update(i);
    }

    // a[p] = x
    void set(int p, S x) {
        p += size;
        for (int i = log; i >= 1; i--) push(p >> i);
        d[p] = x;
        for (int i = 1; i <= log; i++) update(p >> i);
    }

    // returns a[p]
    S get(int p) {
        p += size;
        for (int i = log; i >= 1; i--) push(p >> i);
        return d[p];
    }

    // returns op(a[l], ..., a[r-1]), or e() if l == r
    S prod(int l, int r) {
        if (l == r) return e();
        l += size; r += size;
        // push pending updates on the paths from the root
to l and r-1
        for (int i = log; i >= 1; i--) {
            if (((l >> i) << i) != l) push(l >> i);
            if (((r >> i) << i) != r) push((r - 1) >> i);
        }
        S sml = e(), smr = e(); // left and right
accumulators (op may not be commutative)
        while (l < r) {
            if (l & 1) sml = op(sml, d[l++]);
            if (r & 1) smr = op(d[--r], smr);
            l >>= 1; r >>= 1;
        }
        return op(sml, smr);
    }

    S all_prod() { return d[1]; }

    // a[p] = f(a[p])
    void apply(int p, F f) {
        p += size;
        for (int i = log; i >= 1; i--) push(p >> i);
        d[p] = mapping(f, d[p]);
        for (int i = 1; i <= log; i++) update(p >> i);
    }

    // a[i] = f(a[i]) for all i in [l, r)
    void apply(int l, int r, F f) {
        if (l == r) return;
        l += size; r += size;
        for (int i = log; i >= 1; i--) {
            if (((l >> i) << i) != l) push(l >> i);
            if (((r >> i) << i) != r) push((r - 1) >> i);
        }
        {
            int l2 = l, r2 = r;
            while (l < r) {
                if (l & 1) all_apply(l++, f);
                if (r & 1) all_apply(--r, f);
                l >>= 1; r >>= 1;
            }
            l = l2; r = r2;
        }
        // recompute the ancestors of the touched nodes
        for (int i = 1; i <= log; i++) {
            if (((l >> i) << i) != l) update(l >> i);
            if (((r >> i) << i) != r) update((r - 1) >> i);
        }
    }

    // BINARY SEARCH TO THE RIGHT. Requires g(e()) == true.
    // Returns the largest r such that g(op(a[l], ...,
a[r-1])) is true,
    // assuming g is monotone (true, true, ..., true, false,
false, ...).
    // Returns n if g holds all the way to the end.
    template <class G> int max_right(int l, G g) {
        if (l == _n) return _n;
        l += size;
        for (int i = log; i >= 1; i--) push(l >> i);
        S sm = e();
        do {
            while (l % 2 == 0) l >>= 1;
            if (!g(op(sm, d[l]))) {
                while (l < size) {
                    push(l);
                    l = 2 * l;
                    if (g(op(sm, d[l]))) { sm = op(sm,
d[l]); l++; }
                }
                return l - size;
            }
            sm = op(sm, d[l]);
            l++;
        } while ((l & -l) != l);
        return _n;
    }

    // BINARY SEARCH TO THE LEFT. Requires g(e()) == true.
    // Returns the smallest l such that g(op(a[l], ...,
a[r-1])) is true,
    // assuming g is monotone as l decreases.
    // Returns 0 if g holds all the way to the beginning.
    template <class G> int min_left(int r, G g) {
        if (r == 0) return 0;
        r += size;
        for (int i = log; i >= 1; i--) push((r - 1) >> i);
        S sm = e();
```

```
        do {
            r--;
            while (r > 1 && (r % 2)) r >>= 1;
            if (!g(op(d[r], sm))) {
                while (r < size) {
                    push(r);
                    r = 2 * r + 1;
                    if (g(op(d[r], sm))) { sm = op(d[r],
sm); r--; }
                }
                return r + 1 - size;
            }
            sm = op(d[r], sm);
        } while ((r & -r) != r);
        return 0;
    }

private:
    int _n, size, log;
    vector<S> d;  // d[k]: value of node k (already includes
its own pending update)
    vector<F> lz; // lz[k]: update pending to be pushed to
k's children (internal nodes only)

    void update(int k) { d[k] = op(d[2 * k], d[2 * k +
1]); }
    void all_apply(int k, F f) {
        d[k] = mapping(f, d[k]);
        if (k < size) lz[k] = composition(f, lz[k]);
    }
    void push(int k) {
        all_apply(2 * k, lz[k]);
        all_apply(2 * k + 1, lz[k]);
        lz[k] = id();
    }
};

/* ---------------------------------------------- */
/* Example 1: range add + range sum               */
/* ---------------------------------------------- */
struct SumNode { ll sum; int len; };
SumNode sum_op(SumNode a, SumNode b) { return {a.sum +
b.sum, a.len + b.len}; }
SumNode sum_e() { return {0, 0}; }
SumNode add_mapping(ll f, SumNode x) { return {x.sum + f *
x.len, x.len}; }
ll add_composition(ll f, ll g) { return f + g; }
ll add_id() { return 0; }
using RangeAddSum = LazySegTree<SumNode, sum_op, sum_e, ll,
add_mapping, add_composition, add_id>;

/* ---------------------------------------------- */
/* Example 2: range assign + range min            */
/* ---------------------------------------------- */
const ll NO_ASSIGN = LLONG_MIN; // sentinel: must be a value
never assigned
ll min_op(ll a, ll b) { return min(a, b); }
ll min_e() { return LLONG_MAX; }
ll assign_mapping(ll f, ll x) { return f == NO_ASSIGN ? x :
f; }
ll assign_composition(ll f, ll g) { return f == NO_ASSIGN ?
g : f; } // newer one wins
ll assign_id() { return NO_ASSIGN; }
using RangeAssignMin = LazySegTree<ll, min_op, min_e, ll,
assign_mapping, assign_composition, assign_id>;

int main() {
    // range add + range sum
    vector<ll> A = {18, 17, 13, 19, 15, 11, 20};
    vector<SumNode> init;
    for (ll x : A) init.push_back({x, 1});
    RangeAddSum st(init);
    st.apply(1, 6, 100);              // add 100 to indices
1..5
    cout << st.prod(1, 4).sum << '\n'; // 117 + 113 + 119 =
349
    // first r such that sum(a[0..r]) > 200  (prefix sums:
18, 135, 248, ...)
    int r = st.max_right(0, [](SumNode s) { return s.sum <=
200; });
    cout << r << '\n';                // 2

    // range assign + range min
    RangeAssignMin mn(A);
    mn.apply(2, 5, 7);                // A = [18, 17, 7, 7,
7, 11, 20]
    cout << mn.prod(0, 7) << '\n';    // 7
    cout << mn.prod(5, 7) << '\n';    // 11
    return 0;
}
```

2.8 sparse table

```
// time complexity:
//  - filling DP table: O(N log N)
//  - answering queries: O(1) / O(log N)
// ---------- recursive ---------------
template<class t> struct SparseTable {
    int n; vector<int> memo, log2, *arr;
    SparseTable(vector<int>& _arr) {
        arr = &_arr; n = arr->size(); log2.resize(n+1);
        rep(i,2,n+1) log2[i] = 1 + log2[i >> 1];
        memo.assign(n * (log2[n] + 1), -1);
    }
    int dp(int i, int e) {
        int& ans = memo[e * n + i];
        if (ans != -1) return ans;
        if (e == 0) return ans = (*arr)[i];
        return ans = t::merge(dp(i, e-1), dp(i+(1<<(e-1)),
e-1));
    }
    // option 1: complexity O(1)
    // ** only works if queries can overlap (e.g. max, min,
or, and)
    int query_O1(int l, int r) {
        int e = log2[r - l + 1];
        return t::merge(dp(l,e), dp(r - (1 << e) + 1, e));
    }
    // option 2: complexity O(log N)
    int query_Ologn(int l, int r) {
        int ans = t::neutro;
        int d = r-l+1;
        for (int e = 0; d; e++, d>>=1) {
            if (d & 1) {
                ans = t::merge(ans, dp(l, e));
                l += 1 << e;
            }
        }
        return ans;
    }
};
// ---------- iterative ---------------
template<class t> struct SparseTable {
    int n; vector<int> log2; vector<vector<int>> st;
    SparseTable(vector<int>& arr) {
        log2.resize(n+1); n = arr.size();
        rep(i,2,n+1) log2[i] = 1 + log2[i >> 1];
        int maxe = log2[n];
        st.assign(n, vector<int>(maxe+1));
        rep(i,0,n) st[i][0] = arr[i];
        rep(e,1,maxe+1) {
            int d = 1 << (e-1);
            rep(i,0,n) {
                int j = i + d;
                st[i][e] = j < n ? t::merge(st[i][e-1],
st[j][e-1]) : st[i][e-1];
            }
        }
    }
    // option 1: complexity O(1)
    // ** only works if queries can overlap (e.g. max, min,
or, and)
    int query_O1(int l, int r) {
        int e = log2[r - l + 1];
        return t::merge(st[l][e], st[r - (1 << e) + 1][e]);
    }
    // option 2: complexity O(log N)
    int query_Ologn(int l, int r) {
        int ans = t::neutro;
        int d = r-l+1;
        for (int e = 0; d; e++, d>>=1) {
            if (d & 1) {
                ans = t::merge(ans, st[l][e]);
                l += 1 << e;
            }
```

```
        }
        return ans;
    }
};
struct MIN {
    static const int neutro = INT_MAX;
    static int merge(int x, int y) { return min(x, y); }
};
struct MAX {
    static const int neutro = INT_MIN;
    static int merge(int x, int y) { return max(x, y); }
};
struct OR {
    static const int neutro = 0;
    static int merge(int x, int y) { return x | y; }
};
// example of usage
int main() {
    vector<int> arr = {1, 3, 4, 3, 1, 6, 7, 4, 8, 9};
    SparseTable<MIN> st_min(arr);
    SparseTable<MAX> st_max(arr);
    while (true) {
        int l, r; cin >> l >> r; // read query
        cout << st_min.query_O1(l,r) << '\n'; // print min
        cout << st_max.query_O1(l,r) << '\n'; // print max
    }
    return 0;
}
```

2.9 unionfind

```
struct UnionFind {
    vector<int> p, rank, setSize;
    int numSets;
    UnionFind(int n) {
        numSets = n; setSize.assign(n, 1); rank.assign(n,
0); p.resize(n);
        rep(i,0,n) p[i] = i;
    }
    int findSet(int i) { return (p[i] == i) ? i : (p[i] =
findSet(p[i])); }
    bool isSameSet(int i, int j) { return findSet(i) ==
findSet(j); }
    void unionSet(int i, int j) {
        if (!isSameSet(i, j)) {
            numSets--;
            int x = findSet(i), y = findSet(j);
            // rank is used to keep the tree short
            if (rank[x] > rank[y]) {
                p[y] = x; setSize[x] += setSize[y];
            } else {
                p[x] = y; setSize[y] += setSize[x];
                if (rank[x] == rank[y]) rank[y]++;
            }
        }
    }
    int numDisjointSets() { return numSets; }
    int sizeOfSet(int i) { return setSize[findSet(i)]; }
};
```

3 Strings

3.1 aho corasick

```
/**
 * Aho-Corasick Automaton
 * * Purpose: Efficiently searches for multiple pattern
strings simultaneously within a
 * larger text. It acts as a Trie converted into a
Deterministic Finite Automaton (DFA).
 * * Key Components:
 * - N: The Trie / DFA transition table. N[u][c] moves to
the next state given char 'c'.
 * - L (Suffix Link / Fail Link): Points to the longest
proper suffix of the current
 * state that is also a prefix of some pattern in the Trie.
 * - G (Reverse Suffix Link): The "failure tree", useful for
propagating match counts
 * bottom-up from longest matching suffixes to shorter ones.
 * - E: Stores the count of patterns ending at a specific
node.
 * * Complexity:
 * - Time: $O(\sum |P| + |S|)$ where $P$ are the patterns
and $S$ is the queried text.
 * - Space: $O(M \times A)$ where $M$ is max nodes and $A$
is alphabet size.
 */
struct AC {
    int c = 0, ec = 0, M, A;
    vector<vi> N, G; vi L, E; // L = suffix link, G =
reverse suffix link
    AC (int M, int A) : M(M), A(A), N(M, vi(A, 0)), E(M, 0),
L(M, 0) {}
    int add(const string& s) {
        int p = 0;
        for (char l : s) {
            int t = l - 'a';
            if (!N[p][t]) N[p][t] = ++c;
            p = N[p][t];
        } E[p]++; return p;
    }
    void init() {
        G.assign(M, {});
        queue<int> q; q.push(0); L[0] = -1;
        while (!q.empty()) {
            int p = q.front(); q.pop();
            rep(c, 0, A) {
                int u = N[p][c]; if (!u) continue;
                L[u] = L[p] == -1 ? 0 : N[L[p]][c],
q.push(u);
                G[L[u]].pb(u);
            }
            if (p) rep(c, 0, A) if (!N[p][c]) N[p][c] =
N[L[p]][c];
        }
    }
};
int main() {
    ios::sync_with_stdio(false); cin.tie(0);
    // Initialize AC with max expected nodes (e.g., 100) and
alphabet size (26 for 'a'-'z')
    AC ac(100, 26);
    // Add dictionary words
    ac.add("he");
    ac.add("she");
    ac.add("his");
    ac.add("hers");
    // Build the suffix links and DFA transitions
    ac.init();
    // Query text: "ushers"
    string text = "ushers";
    int current_state = 0;
    int total_matches = 0;
    cout << "Matches found at ending indices:\n";
    for (int i = 0; i < text.length(); ++i) {
        current_state = ac.N[current_state][text[i] - 'a'];
        // Follow suffix links to count all matching sub-
patterns ending here
        int temp = current_state;
        while (temp > 0) {
            if (ac.E[temp] > 0) {
                cout << "- Found a match ending at index "
<< i << "\n";
                total_matches += ac.E[temp];
            }
            temp = ac.L[temp];
        }
    }
    cout << "Total patterns matched: " << total_matches <<
"\n";
    return 0;
}
```

3.2 kmp

```
// Build longest proper prefix/suffix array (lps) for
pattern
// lps[i] = length of the longest proper prefix which is
also suffix in pattern[0 .. i]
void init_lps(string& pattern, int lps[]) {
    int n = pattern.size();
    lps[0] = 0; // base case: no proper prefix/suffix for
pattern[0 .. 0] (length 1)
    rep(j, 1, n) { // for each pattern[0 .. j]
```

```
        int i = lps[j-1]; // i points to the char next to
lps of previous iteration
        while (pattern[i] != pattern[j] and i > 0) i =
lps[i-1];
        lps[j] = pattern[i] == pattern[j] ? i+1 : 0;
    }
}

// Count number of matches of pattern string in target
string using KMP algorithm
int count_matches(string& pattern, string& target) {
    int n = pattern.size(), m = target.size();
    int lps[n];
    init_lps(pattern, lps); // build lps array
    int matches = 0;
    int i = 0; // i tracks current char in pattern to
compare
    rep(j, 0, m) { // j tracks each char in target to
compare
        // try to keep prefix before i as long as possible
while ensuring i matches j
        while (pattern[i] != target[j] and i > 0) i =
lps[i-1];
        if (pattern[i] == target[j]) {
            if (++i == n) { // we matched the whole pattern
                i = lps[n-1]; // shift the pattern so that
the longest proper prefix/suffix pair is aligned
                matches++;
            }
        }
    }
    return matches;
}

int main() { // usage
    string target, pattern;
    while (true) {
        cin >> target >> pattern;
        cout << count_matches(pattern, target) << "
matches\n";
    }
    return 0;
}
```

3.3 rolling hashing

```
/* ======================================
 * POLYNOMIAL ROLLING HASH MATH REMINDER
 * --------------------------------------
 * A string is treated as a polynomial evaluated at a
specific Base (B),
 * modulo a Prime (P).
 * * 1. Prefix Hash Definition:
 * H[k] = (s[0]*B^k + s[1]*B^(k-1) + ... + s[k]*B^0) % P
 * * 2. Iterative Construction (used in init):
 * H[k] = (H[k-1] * B + s[k]) % P
 * * 3. Substring Hash Extraction for s[i..j] (used in
hash(i, j)):
 * Hash(i, j) = (H[j] - H[i-1] * B^(length)) % P
 * = (H[j] + P - (H[i-1] * B^(j - i + 1)) % P) % P
 * * Conceptually, to get the hash of s[i..j], we take the
hash of the prefix
 * ending at j, and "subtract" the prefix ending at i-1. To
align the
 * "powers" of B, we must multiply H[i-1] by B raised to the
substring length.
 * ====================================== */
const int MAXLEN = 1e6;
// ----------------------------------------
// Rolling Hashing: double hash (extra safety)
struct RH_double { // rolling hashing
    static const ull B = 131; // base
    static const ull P[2]; // primes
    static ull pow[2][MAXLEN];
    static ull add(ull x, ull y, int a) { return (x + y) %
P[a]; }
    static ull mul(ull x, ull y, int a) { return (x * y) %
P[a]; }
    // Initializes powers of B. Call RH_double::init() once
globally.
    static void init(int a) {
        pow[a][0] = 1;
        rep(i, 1, MAXLEN) pow[a][i] = mul(B, pow[a][i-1],
a);
    }
    static void init() { init(0); init(1); }
    // Mathematically shifts the hash 'h' left by 'k'
positions.
    // Equivalent to appending 'k' virtual characters
(zeros) to the right
    // of the string represented by 'h'. Useful for
concatenating hashes.
    static ull shift_left(ull h, int k) {
        ull h0 = h >> 32;                       // a=0 is in
the upper 32 bits
        ull h1 = h & ((1ull << 32) - 1);        // a=1 is in
the lower 32 bits
        h0 = mul(h0, pow[0][k], 0);
        h1 = mul(h1, pow[1][k], 1);
        return (h0 << 32) | h1;
    }
    // Adds two packed hashes together
    static ull add(ull x, ull y) {
        ull x0 = x >> 32, x1 = x & ((1ull << 32) - 1);
        ull y0 = y >> 32, y1 = y & ((1ull << 32) - 1);
        ull ans0 = add(x0, y0, 0);
        ull ans1 = add(x1, y1, 1);
        return (ans0 << 32) | ans1;
    }
    vector<ull> h[2];
    int len;
    // s must be 0-indexed [0, s.size()-1]
    void init(const vector<int>& s) {
        // for (int x : s) assert (x >= 1); // DEBUGGING
        len = s.size();
        if (len == 0) return;
        rep(a, 0, 2) {
            h[a].resize(len);
            h[a][0] = s[0];
            rep(i, 1, len) h[a][i] = add(mul(h[a][i-1], B,
a), s[i], a);
        }
    }
    RH_double(const vector<int>& s) { init(s); } // from
vector<int>
    RH_double(const string& s, char ref = 'a') { // from
string
        vector<int> tmp;
        for (char c : s) tmp.push_back(c - ref + 1);
        init(tmp);
    }
    // Returns packed hash of substring s[i..j] (both i and
j are INCLUSIVE)
    ull hash(int i, int j, int a) {
        if (i == 0) return h[a][j];
        return add(h[a][j], P[a] - mul(h[a][i-1], pow[a][j-
i+1], a), a);
    }
    // Returns packed double hash of substring s[i..j]
(INCLUSIVE bounds)
    ull hash(int i, int j) { return (hash(i, j, 0) << 32) |
hash(i, j, 1); }
    // Returns hash of the entire string s[0..len-1]
(INCLUSIVE bounds)
    ull hash() { return hash(0, len - 1); }
};

// these lines are necessary for the code to compile
const ull RH_double::P[2] = {(ull)1e9+21, (ull)1e9+9};
ull RH_double::pow[2][MAXLEN];

// ----- usage & testing
int main() {
    RH_double::init();
    string s = "abacaba";
    int l1 = 0, r1 = 2; // "aba"
    int l2 = 4, r2 = 6; // "aba"
    char cmin = *min_element(s.begin(), s.end());
    RH_double rh(s, cmin); // O(s.size())
    ull h1 = rh.hash(l1, r1); // O(1)
    ull h2 = rh.hash(l2, r2); // O(1)
    string s1 = s.substr(l1, r1 - l1 + 1);
    string s2 = s.substr(l2, r2 - l2 + 1);
    cout << "Strings s1=" << s1 << " and s2=" << s2 << " are
" << (h1 == h2 ? "EQUAL" : "DISTINCT") << endl;
```

```
    return 0;
}
```

3.4 shortest repeating cycle

```
int shortest_repeating_cycle(string& seq) {
    // KMP : lps step
    int n = seq.size();
    int lps[n];
    lps[0] = 0;
    int i = 0, j = 1;
    while (j < n) {
        while (i > 0 and seq[i] != seq[j]) i = lps[i-1];
        if (seq[i] == seq[j]) lps[j] = ++i;
        else lps[j] = 0;
        j++;
    }
    int len = n - lps[n-1];
    return (n % len) ? n : len;
}
// test
int main() {
    string line; cin >> line;
    int cycle = shortest_repeating_cycle(line);
    cout << line.substr(0, cycle) << endl;
    return 0;
}
```

3.5 suffix array

```
// ======================================================
// Suffix Array Construction : Prefix Doubling + Radix Sort
// ======================================================
// Complexity: O(N*log(N))
// references: https://www.cs.helsinki.fi/u/tpkarkka/opetus/10s/spa/lecture11.pdf
// https://youtu.be/_TUeAdu-U_k
struct SA {
    int n; vector<int> counts, rank, rank_, sa, sa_, lcp; // lcp is optional
    inline int gr(int i) { return i < n ? rank[i]: 0; }
    void csort(int maxv, int k) {
        counts.assign(maxv+1, 0);
        rep(i,0,n) counts[gr(i+k)]++;
        rep(i,1,maxv+1) counts[i] += counts[i-1];
        invrep(i,n-1,0) sa_[--counts[gr(sa[i]+k)]] = sa[i];
        sa.swap(sa_);
    }
    void get_sa(vector<int>& s) {
        rep(i,0,n) sa[i] = i;
        sort(sa.begin(), sa.end(), [&s](int i, int j) { return s[i] < s[j]; });
        int r = rank[sa[0]] = 1;
        rep(i,1,n) rank[sa[i]] = (s[sa[i]] != s[sa[i-1]]) ? ++r : r;
        for (int h=1; h < n and r < n; h <<= 1) {
            csort(r, h); csort(r, 0); r = rank_[sa[0]] = 1;
            rep(i,1,n) {
                if (rank[sa[i]] != rank[sa[i-1]] or
                    gr(sa[i]+h) != gr(sa[i-1]+h)) ++r;
                rank_[sa[i]] = r;
            } rank.swap(rank_);
        }
    }
    // LCP construction in O(N) using Kasai's algorithm
    // reference: https://codeforces.com/blog/entry/12796?#comment-175287
    void get_lcp(vector<int>& s) { // lcp is optional
        lcp.assign(n, 0); int k = 0;
        rep(i,0,n) {
            int r = rank[i]-1;
            if (r == n-1) { k = 0; continue; }
            int j = sa[r+1];
            while (i+k<n and j+k<n and s[i+k] == s[j+k]) k++;
            lcp[r] = k;
            if (k) k--;
        }
    }
    SA(vector<int>& s) {
        n = s.size();
        rank.resize(n); rank_.resize(n);
        sa.resize(n); sa_.resize(n);
        get_sa(s); get_lcp(s); // lcp is optional
    }
};

int main() { // how to use
    string test; cin >> test;
    vector<int> s;
    for (char c : test) s.push_back(c);
    SA sa(s);
    for (int i : sa.sa) cout << i << ":\t" << test.substr(i) << '\n';
    rep(i,0,s.size()) {
        printf("LCP between %d and %d is %d\n", i, i+1, sa.lcp[i]);
    }
}
```

3.6 trie

```
struct Trie {
    vector<vector<int>> g;
    vector<int> count;
    int vocab;
    Trie(int vocab, int maxdepth = 10000) : vocab(vocab) {
        g.reserve(maxdepth);
        g.emplace_back(vocab, -1);
        count.reserve(maxdepth);
        count.push_back(0);
    }
    int move_to(int u, int c) {
        // assert (0 <= c and c < vocab); // debugging
        int& v = g[u][c];
        if (v == -1) {
            v = g.size();
            g.emplace_back(vocab, -1);
            count.push_back(0);
        }
        count[v]++;
        return v;
    }
    void insert(const string& s, char ref = 'a') {  // insert string
        int u = 0; count[0]++; for (char c : s) u = move_to(u, c - ref);
    }
    void insert(vector<int>& s) { // insert vector<int>
        int u = 0; count[0]++; for (int c : s) u = move_to(u, c);
    }
    int size() { return g.size(); }
};

int main() { // example of usage
    Trie trie(26);
    for (string s : {"hell", "hello", "hellyeah", "helpzzzz", "abcdefg"}) {
        cout << "inserting " <<  s << '\n';
        trie.insert(s);
        cout << "\ttrie size = " << trie.size() << '\n';
    }
    return 0;
}
```

4 Divide&conquer

4.1 mergesort

```
void mergesort_(vector<int>& a, int i, int j, vector<int>& b) {
    if (i == j) return; // base case
    // general case
    int m = (i+j)/2;
    mergesort_(a, i, m, b);
    mergesort_(a, m+1, j, b);
    // merge efficiently
    int r = i, s = m+1;
    int k = i;
    while (r <= m and s <= j) b[k++] = a[r] <= a[s] ? a[r++] : a[s++];
    while (r <= m) b[k++] = a[r++];
    while (s <= j) b[k++] = a[s++];
```

```
    for (int x = i; x <= j; ++x) a[x] = b[x];
}

void mergesort(vector<int>& a) {
    vector<int> b(a.size());
    mergesort_(a, 0, a.size()-1, b);
}
```

5 Mathematics

5.1 binary modular exponentiation

```
// compute a^b (mod m)
// use ll: with m ~ 1e9, a * a can reach ~1e18, which
overflows int
ll binary_exp(ll a, ll b, ll m) {
    a %= m;
    ll res = 1;
    while (b > 0) {
        if (b&1) res = (res * a) % m;
        a = (a * a) % m;
        b >>= 1;
    }
    return res;
}
```

5.2 chinese remainder theorem

```
inline ll mod(ll x, ll m) { return ((x %= m) < 0) ? x+m :
x; }
inline ll mul(ll x, ll y, ll m) { return (x * y) % m; }
inline ll add(ll x, ll y, ll m) { return (x + y) % m; }

// extended euclidean algorithm
// finds g, x, y such that
//    a * x + b * y = g = GCD(a,b)
ll gcdext(ll a, ll b, ll& x, ll& y) {
    ll r2, x2, y2, r1, x1, y1, r0, x0, y0, q;
    r2 = a, x2 = 1, y2 = 0;
    r1 = b, x1 = 0, y1 = 1;
    while (r1) {
        q = r2 / r1;
        r0 = r2 % r1;
        x0 = x2 - q * x1;
        y0 = y2 - q * y1;
        r2 = r1, x2 = x1, y2 = y1;
        r1 = r0, x1 = x0, y1 = y0;
    }
    ll g = r2; x = x2, y = y2;
    if (g < 0) g = -g, x = -x, y = -y; // make sure g > 0
    // for debugging (in case you think you might have bugs)
    // assert (g == a * x + b * y);
    // assert (g == __gcd(abs(a),abs(b)));
    return g;
}

// ================================================
// CRT for a system of 2 modular linear equations
// ================================================
// We want to find X such that:
//   1) x = r1 (mod m1)
//   2) x = r2 (mod m2)
// The solution is given by:
//    sol = r1 + m1 * (r2-r1)/g * x' (mod LCM(m1,m2))
// where x' comes from
//   m1 * x' + m2 * y' = g = GCD(m1,m2)
//   where x' and y' are the values found by extended
euclidean algorithm (gcdext)
// Useful references:
//   https://codeforces.com/blog/entry/61290
//   https://forthright48.com/chinese-remainder-theorem-
part-1-coprime-moduli
//   https://forthright48.com/chinese-remainder-theorem-
part-2-non-coprime-moduli
// ** Note: this solution works if lcm(m1,m2) fits in a long
long (64 bits)
pair<ll,ll> CRT(ll r1, ll m1, ll r2, ll m2) {
    ll g, x, y; g = gcdext(m1, m2, x, y);
    if ((r1 - r2) % g != 0) return {-1, -1}; // no solution
    ll z = m2/g;
    ll lcm = m1 * z;
    ll sol = add(mod(r1, lcm), m1*mul(mod(x,z),mod((r2-r1)/
g,z),z), lcm);
    // for debugging (in case you think you might have bugs)
    // assert (0 <= sol and sol < lcm);
    // assert (sol % m1 == r1 % m1);
    // assert (sol % m2 == r2 % m2);
    return {sol, lcm}; // solution + lcm(m1,m2)
}

// ================================================
// CRT for a system of N modular linear equations
// ================================================
//  Args:
//      r = array of remainders
//      m = array of modules
//      n = length of both arrays
//  Output:
//      a pair {X, lcm} where X is the solution of the
sytemm
//          X = r[i] (mod m[i]) for i = 0 ... n-1
//      and lcm = LCM(m[0], m[1], ..., m[n-1])
//      if there is no solution, the output is {-1, -1}
// ** Note: this solution works if LCM(m[0],...,m[n-1]) fits
in a long long (64 bits)
pair<ll,ll> CRT(ll* r, ll* m, int n) {
    ll r1 = r[0], m1 = m[0];
    rep(i,1,n) {
        ll r2 = r[i], m2 = m[i];
        ll g, x, y; g = gcdext(m1, m2, x, y);
        if ((r1 - r2) % g != 0) return {-1, -1}; // no
solution
        ll z = m2/g;
        ll lcm = m1 * z;
        ll sol = add(mod(r1, lcm), m1*mul(mod(x,z),mod((r2-
r1)/g,z),z), lcm);
        r1 = sol;
        m1 = lcm;
    }
    // for debugging (in case you think you might have bugs)
    // assert (0 <= r1 and r1 < m1);
    // rep(i,0,n-1) assert (r1 % m[i] == r[i]);
    return {r1, m1};
}
```

5.3 euclidean algorithm

```
inline ll mod(ll x, ll m) { return ((x %= m) < 0) ? x+m :
x; }

/* ============================ */
/* GCD (greatest common divisor) */
/* ============================ */
// OPTION 1: using C++ builtin function __gcd
__gcd(a,b)
// OPTION 2: manually usings euclid's algorithm
int gcd (ll a, ll b) {
    while (b) { a %= b; swap(a,b); }
    return a;
}

/* ============ */
/* extended GCD */
/* ============ */
// extended euclid's algorithm: find g, x, y such that
// a * x + b * y = g = gcd(a, b)
// The algorithm finds a solution (x0,y0) but there are
infinite more:
//   x(n) = x0 + n * (b/g)
//   y(n) = y0 - n * (a/g)
// where n is integer, are the set of all solutions

// --- version 1: iterative
ll gcdext(ll a, ll b, ll& x, ll& y) {
    ll r2, x2, y2, r1, x1, y1, r0, x0, y0, q;
    r2 = a, x2 = 1, y2 = 0;
    r1 = b, x1 = 0, y1 = 1;
    while (r1) {
        q = r2 / r1;
        r0 = r2 % r1;
        x0 = x2 - q * x1;
        y0 = y2 - q * y1;
        r2 = r1, x2 = x1, y2 = y1;
        r1 = r0, x1 = x0, y1 = y0;
```

```
    }
    ll g = r2; x = x2, y = y2;
    if (g < 0) g = -g, x = -x, y = -y; // make sure g > 0
    // for debugging (in case you think you might have bugs)
    // assert (g == a * x + b * y);
    // assert (g == __gcd(abs(a),abs(b)));
    return g;
}

// --- version 2: recursive
ll gcdext(ll a, ll b, ll& x, ll& y) {
    if (a == 0) {
        x = 0, y = 1;
        return b;
    }
    ll x1, y1;
    ll g = gcdext(b % a, a, x1, y1);
    x = y1 - (b / a) * x1;
    y = x1;
    return g;
}

/* ====================== */
/* multiplicative inverse */
/* ====================== */
// find x such that a * x = 1 (mod m)
// an inverse exists iff gcd(a, m) == 1

// --- option 1: extended euclid (works for ANY modulus m)
// this is the same as finding x, y such that
// a * x + m * y = 1, which can be done with gcdext
// and then returning x (mod m)
ll mulinv(ll a, ll m) {
    ll x, y;
    if (gcdext(a, m, x, y) == 1) return mod(x, m); // make
sure 0 <= x < m
    return -1; // no inverse exists
}

// --- option 2: Fermat's little theorem (ONLY when the
modulus p is PRIME)
// Fermat: if p is prime and a is not a multiple of p, then
//   a^(p-1) = 1 (mod p)
// dividing both sides by a:
//   a^(p-2) = a^(-1) (mod p)
// so the inverse is just a fast power, O(log p).
// Typical use: p = 1e9+7 or p = 998244353 (both prime).
// WARNING: if a is a multiple of p (e.g. a = 0), no inverse
exists and this
// silently returns 0. When inverting n!, this is safe as
long as n < p.
// binary_exp: see binary_modular_exponentiation.cpp
ll mulinv_fermat(ll a, ll p) {
    return binary_exp(mod(a, p), p - 2, p);
}
```

```
/* ========================== */
/* Linear Diophantine Equation */
/* ========================== */
// recommended readings:
// http://gauss.math.luc.edu/greicius/Math201/Fall2012/
Lectures/linear-diophantine.article.pdf
// http://mathonline.wikidot.com/solutions-to-linear-
diophantine-equations

// find intengers x and y such that a * x + b * y = c
bool lindiopeq(ll a, ll b, ll c, ll& x, ll& y) {
    if (a == 0 and b == 0) { // special case
        if (c == 0) { x = y = 0; return true; }
        return false;
    }
    // general case
    ll s, t;
    ll g = gcdext(a,b,s,t);
    if (c % g == 0) {
        x = s*(c/g), y = t*(c/g);
        return true;
    }
    return false;
}

/* ========================== */
/* Linear Congruence Equation */
/* ========================== */
// recommended reading:
// http://gauss.math.luc.edu/greicius/Math201/Fall2012/
Lectures/linear-congruences.article.pdf

// find smallest integer x (mod m) that solves the equation
//   a * x = b (mod m)
bool lincongeq(ll a, ll b, ll m, ll& x) {
    assert (m > 0);
    a = mod(a,m);
    b = mod(b,m);
    ll s, t;
    ll g = gcdext(a,m,s,t);
    if (b % g == 0) {
        ll bb = b/g;
        ll mm = m/g;
        ll n = -s*bb/mm;
        x = s*bb + n*mm;
        if (x < 0) x += mm;
        // for debugging
        // assert (0 <= x and x < m);
        // assert (mod(a*x,m) == b);
        return true;
    }
    return false;
}
```

5.4 integer root square

```
// using sqrt()
bool perfect_square(ll x, ll& root) {
  if (x < 0) return false;
  root = (ll)sqrt(x);
  return (root * root == x || ++root * root == x);
}

// Newton's method
ll isqrt(ll x) {
  ll y0 = x;
  while (true) {
    ll y1 = (y0 +  x / y0) / 2;
    if (y1 == y0) break;
    y0 = y1;
  }
  return y0;
}
bool isPerfectSquare(ll x, ll& root) {
  root = isqrt(x);
  return root * root == x;
}
```

5.5 modular binomial coefficient

```
const ll MOD = 1000000007ll; // a prime number
const int MAXN = 1000;

/* ================== */
/*  MODULAR BINOMIAL  */
/* ================== */
// choose_mod(n,k) = n! / (k! * (n-k)!) % MOD

// ---------------------
// method 1: DP
// choose(n,k) = (choose(n-1,k-1) + choose(n-1,k)) % MOD
// choose(n,0) = choose(n,n) = 1

// 1.1) DP top-down
ll memo[MAXN+1][MAXN+1];
ll choose(int n, int k) {
    ll& ans = memo[n][k];
    if (ans != -1) return ans;
    if (k == 0) return ans = 1;
    if (n == k) return ans = 1;
    if (n < k) return ans = 0;
    return ans = (choose(n-1,k) + choose(n-1,k-1)) % MOD;
}

// 1.2) DP bottom-up
ll choose[MAXN+1][MAXN+1];
rep(m,1,MAXN+1) {
    choose[m][0] = choose[m][m] = 1;
    rep(k,1,m) choose[m][k] = (choose[m-1][k] + choose[m-1]
```

```
[k-1]) % MOD;
}

// ------------------------------------------------
// method 2: factorials + inverse factorials (RECOMMENDED for large n, MOD prime)
// precompute in O(MAXN), then each query is O(1), with no MAXN x MAXN table,
// so it works for MAXN ~ 1e6 or more.
//   fac[i]     = i! (mod MOD)
//   inv_fac[i] = (i!)^-1 (mod MOD)
//   choose(n,k) = fac[n] * inv_fac[k] * inv_fac[n-k] (mod MOD)
// only ONE modular inverse is needed (of MAXN!), via Fermat's little theorem
// (mulinv_fermat in euclidean_algorithm.cpp). Requires MAXN < MOD, so that
// MAXN! is not a multiple of MOD.
// The rest are filled backwards, since (i-1)! = i! / i:
//   1 / (i-1)! = i * (1 / i!)  =>  inv_fac[i-1] = inv_fac[i] * i
ll fac[MAXN+1], inv_fac[MAXN+1];
void init() {
    fac[0] = 1;
    rep(i,1,MAXN+1) fac[i] = fac[i-1] * i % MOD;
    inv_fac[MAXN] = binary_exp(fac[MAXN], MOD-2, MOD); // Fermat
    invrep(i,MAXN,1) inv_fac[i-1] = inv_fac[i] * i % MOD;
}
ll choose(int n, int k) {
    if (k < 0 or k > n) return 0;
    return fac[n] * inv_fac[k] % MOD * inv_fac[n-k] % MOD;
}

// ------------------------------------------------
// method 3: factorials and multiplicative inverse
// n! / (k! * (n-k)!) =  n! * (k! * (n-k)!)^-1  (MOD N)
// we need to find the multiplicative inverse of (k! * (n-k)!) MOD N

ll fac[MAXN+1];
ll choose_memo[MAXN+1][MAXN+1];
void init() {
    fac[0] = 1;
    rep(i,1,MAXN+1) fac[i] = (i * fac[i-1]) % MOD;
    memset(choose_memo, -1, sizeof choose_memo);
}
ll choose_mod(int n, int k) {
    if (choose_memo[n][k] != -1) return choose_memo[n][k];
    return choose_memo[n][k] = mul(fac[n], mulinv(mul(fac[k], fac[n-k])));
}
```

5.6 modular fibonacci

```
//=========================================================
// Modular Fibonacci with (Modular) Matrix Exponentiation
//=========================================================
//source: http://mathoverflow.net/questions/40816/fibonacci-series-mod-a-number
const ull MOD = 1000000000;

vector<ull> mult(const vector<ull>& A, const vector<ull>& B)
{
  vector<ull> res
  { (((A[0] * B[0]) % MOD) + ((A[1] * B[2]) % MOD)) % MOD, //m11
    (((A[0] * B[1]) % MOD) + ((A[1] * B[3]) % MOD)) % MOD, //m12
    (((A[2] * B[0]) % MOD) + ((A[3] * B[2]) % MOD)) % MOD, //m21
    (((A[2] * B[1]) % MOD) + ((A[3] * B[3]) % MOD)) % MOD // m22
  };
  return res;
}

vector<ull> raise(const vector<ull>& matrix, ull exp) {
  if (exp == 1)
    return matrix;
  ull m = exp / 2;
  vector<ull> A = raise(matrix, m);
  if (exp % 2 == 0)
    return mult(A, A);
  else
    return mult(mult(A, A), matrix);
}

int main() {
  int P, k;
  ull y;
  scanf("%d", &P);
  vector<ull> fib_matrix { 1, 1, 1, 0 }; //starting fibonacci matrix [f2, f1, f1, f0]
  while (P-- > 0) {
    scanf("%d %llu", &k, &y);
    vector<ull> ansm = raise(fib_matrix, y);
    ull ans = ansm[1];
    printf("%d %llu\n", k, ans);
  }
  return 0;
}
```

5.7 modular multinomial coefficient

```
const ll MOD = 1000000007ll; // a prime number
const int MAXN = 1000;

/* ==================== */
/*  MODULAR MULTINOMIAL  */
/* ==================== */
ll memo[MAXN+1][MAXN+1];
ll choose(int n, int k) {
    ll& ans = memo[n][k];
    if (ans != -1) return ans;
    if (k == 0) return ans = 1;
    if (n == k) return ans = 1;
    if (n < k) return ans = 0;
    return ans = (choose(n-1,k) + choose(n-1,k-1)) % MOD;
}
// reference: https://math.stackexchange.com/a/204209/503889
ll multinomial(vector<int> ks) {
    int n = 0;
    ll ans = 1;
    for (int k : ks) {
        n += k;
        ans = (ans * choose(n,k)) % MOD;
    }
    return ans;
}
```

5.8 primality test

```
// ===============
// trial division
//=================
// complexity: ~O( sqrt(x) )
bool isPrime(int x) {
    for (int d = 2; d * d <= x; d++) {
        if (x % d == 0)
            return false;
    }
    return true;
}
// ======================================
// trial division with precomputed primes
// ======================================
// complexity: ~O( sqrt(x)/log(sqrt(x)) )
//   + time of precomputing primes
bool isPrime(int x, vector<int>& primes) {
    for (int p : primes) {
        if (p*p > x) break;
        if (p % x == 0)
            return false;
    }
    return true;
}
// =============
// Miller-Rabin
// =============
// complexity: O (k * log^3(n))
// references:
// https://cp-algorithms.com/algebra/primality_tests.html
```

```cpp
// https://en.wikipedia.org/wiki/Miller%E2%80%93Rabin_primality_test#Complexity
using u64 = uint64_t;
using u128 = __uint128_t;
u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}
bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};
bool MillerRabin(u64 n) { // returns true if n is probably prime, else returns false.
    if (n < 4) return n == 2 || n == 3;
    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        s++;
    }
    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (check_composite(n, a, d, s))
            return false;
    }
    return true;
}
bool MillerRabin(u64 n) { // returns true if n is prime, else returns false.
    if (n < 2) return false;
    int r = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }
    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
            return false;
    }
    return true;
}
```

5.9 prime factorization

```cpp
//====================
// Prime Factorization
//====================
// reference: https://cp-algorithms.com/algebra/factorization.html
// Method 1: trial division
// complexity: ~ O( sqrt(n) + log_2(n) )
vector<int> trial_division(int n) {
    vector<int> factors;
    for (int d = 2; d*d <= n; d++) {
        while (n % d == 0) {
            factors.push_back(d);
            if ((n /= d) == 1) return factors;
        }
    }
    if (n > 1) factors.push_back(n);
    return factors;
}
// Method 2: precomputed primes
// complexity: ~ O( sqrt(n) / log(sqrt(n)) + log_2(n) )
//   + time of precomputing primes
vector<int> trial_division_precomp(int n, vector<int>& primes) {
    vector<int> factors;
    for (int d : primes) {
        if (d*d > n) break;
        while (n % d == 0) {
            factors.push_back(d);
            if ((n /= d) == 1) return factors;
        }
    }
    if (n > 1) factors.push_back(n);
    return factors;
}
//====================================
// Prime Factorization of Factorials
//====================================
// references:
// http://mathforum.org/library/drmath/view/67291.html
// https://janmr.com/blog/2010/10/prime-factors-of-factorial-numbers/
#define umap unordered_map
umap<int,int> factorial_prime_factorization(int n, vector<int>& primes) {
    umap<int,int> prime2exp;
    for (int p : primes) {
        if (p > n) break;
        int e = 0;
        int tmp = n;
        while ((tmp /= p) > 0) e += tmp;
        if (e > 0) prime2exp[p] = e;
    }
    return prime2exp;
}
```

5.10 sieve of eratosthenes

```cpp
vector<int> get_primes_up_to(int n) { // O(n log log n)
    vector<bool> is_prime(n + 1, true);
    int limit = floor(sqrt(n));
    rep (i,2,limit+1)
        if (is_prime[i])
            for (int j = i*i; j <= n; j += i)
                is_prime[j] =  false;
    vector<int> primes;
    rep(i,2,n) if (is_prime[i]) primes.push_back(i);
    return primes;
}
```

6 Dynamic_programming

6.1 divide&conqueroptimization

```cpp
#define MAXG 1000
#define MAXL 1000
int G,L;
ll DP[MAXG+1][MAXL+1];

// return cost of forming a group with items in the range i .. j
ll group_cost(int i, int j) { ... }

/**
 Calculates the values of DP[g][l] for l1 <= l <= l2 (a range of cells in row 'g')
 using divide & conquer optimization

 DP[g][l] means: given a list of the first 'l' items, partition them into 'g' groups,
 each group consisting of consecutive items (left to right), so that the total
 cost of forming those groups is the minimum possible.

 If we form one group at a time, from right to left, this leads to the following
 recursion:

 DP[g][l] = min { DP[g-1][k] + group_cost(k,l-1) for k = g-1 .. l-1 }
 DP[1][l] = group_cost(0, l-1)
```

```
 in other words:

 DP[g][l] = DP[g-1][best_k] + group_cost(best_k,l-1)
    where best_k is the left most value of k where the
minimum is reached

Now, for a given 'g':

    If best_k(g,0) <= best_k(g,1) <= best_k(g,2) <= ... <=
best_k(g,L-1) holds

    Then, we can propagate those best_k's recursively to
reduce the range of
    candidate k's for each DP[g][l] problem we solve.
    Using Divide & Conquer, we fill the whole row 'g'
recursively with
    recursion depth O(log(L)), and each recursion layer
taking O(L) time.

Doing this for G groups, the total computation cost is
O(G*L*log(L))

*/
void fill_row(int g, int l1, int l2, int k1, int k2) {
    if (l1 > l2) return; // ensure valid range
    int lm = (l1+l2)/2; // solve middle case
    int kmin = max(g-1, k1);
    int kmax = min(lm-1, k2);
    int best_k = -1;
    ll mincost = LLONG_MAX;
    rep(k,kmin,kmax+1) {
        ll tmp = DP[g-1][k] + group_cost(k, lm-1);
        if (mincost > tmp) mincost = tmp, best_k = k;
    }
    DP[g][lm] = mincost;
    fill_row(g, l1, lm-1, k1, best_k); // solve left cases
    fill_row(g, lm+1, l2, best_k, k2); // solve right cases
}

void fill_dp() {
    // base: g = 1
    rep(l,1,L+1) DP[1][l] = group_cost(0,l-1);
    // other: g >= 2
    rep(g,2,G+1) fill_row(g,g,L,0,L);
}
```

6.2 lis

```
// ====================================
// LIS (Longest Increasing Subsequence)
// ====================================
// references:
// https://stackoverflow.com/questions/2631726/how-to-
determine-the-longest-increasing-subsequence-using-dynamic-
programming
const int MAXLEN = 1000000;
// return the length of the longest increasing (non-
decreasing)
// subsequence in values
int LIS(vector<int>& values) {
    static int q[MAXLEN+1];
    int len = 0;
    q[0] = -INT_MAX; // make sure it's strictly smallest
    for (int val : values) {
        if (q[len] < val) { // use <= if non-decreasing
            q[++len] = val;
        } else {
            int l=1, r=len;
            while (l<r) {
                int m = (l+r)>>1;
                if (q[m] >= val) { // use > if non-
decreasing
                    r = m;
                } else {
                    l = m+1;
                }
            }
            q[l] = val;
        }
    }
    return len;
}
```

6.3 tsp

```
// ---------------------------------
// Travelling Salesman Problem (TSP)
// ---------------------------------
// complexity: O(2^N * N)

const int MAXN = 14; // maximum number of nodes in the
problem statement
int cost[MAXN][MAXN]; // cost[i][j]: cost to travel from
node i to node j
// make sure cost[i][j] >= 0
int start_index; // OPTIONAL: if you need to remember the
start node

// dp(mask, i): find the minimum cost of visiting all nodes
indicated by 'mask'
// starting from node 'i'.
//  *** OPTIONAL VARIANT: include the cost of returning back
to the start node at the end
//
//   * mask: an int whose bits indicate the nodes we want to
visit
//      ** if j-th bit in mask is 1, the j-th node should be
visited
//         else, the j-th node should be ignored
//   * i: node we are starting the travel from (i'th bit
should be 1 in mask)
int memo[1 << MAXN][MAXN]; // 2^MAXN x MAXN
int dp(int mask, int i) {
    // base case 1: problem already solved
    int& ans = memo[mask][i];
    if (ans != -1) return ans;
    // mark i-th node as visited
    int mask2 = mask & ~(1 << i);
    // base case 2: nothing else to visit
    if (mask2 == 0) return ans = 0;
    // if (mask2 == 0) return ans = cost[i][start_index]; //
<--- if returning back to start
    // general case: try all possible next nodes
    ans = INT_MAX;
    for (int j = 0, tmp=mask2; tmp; ++j, tmp>>=1) {
        if (tmp & 1) ans = min(ans, cost[i][j] + dp(mask2,
j));
    }
    // return answer
    return ans;
}

int main() { // usage
    memset(memo, -1, sizeof memo);
    start_index = 0;
    cout << dp((1 << N)-1, start_index); // <-- mincost of
visiting all N nodes starting from 0
}
```

7 Search

7.1 binarysearch

```
// Find the index of the first item that satisfies a
predicate
// over a range [i,j), i.e., from i to j-1
// If no such index exists, j is returned
template<typename T, typename Predicate>
int binsearch(const std::vector<T>& array, int i, int j,
Predicate predicate) {
    // assert(i < j); // since the range is [i,j), then j
must be > i
    while (i < j) {
        int m = (i + j) / 2;
        if (predicate(array[m])) j = m;
        else i = m + 1;
    }
    return i; // notice that i == j if the predicate is
false for the whole range
}
// Example usage with a predicate function
vector<int> array = {1, 2, 3, 4, 5};
int predicate(int x) { return x >= 3; }
```

```cpp
int index = binsearch(array, 0, array.size(), predicate);

// ----------------------------
// EXAMPLE 1: Integer Lowerbound
// predicate(a, i, key) = (a[i] >= key)
// i.e. "first element >= key"
int lowerbound(vector<int>& a, int key, int i, int j) {
    while (i < j) {
        int m = (i + j) / 2;
        if (a[m] >= key) j = m;
        else i = m + 1;
    }
    return i;
}

// ----------------------------
// EXAMPLE 2: Integer Upperbound
// predicate(a, i, key) = (a[i] > key)
// i.e. "first element > key"
int upperbound(vector<int>& a, int key, int i, int j) {
    while (i < j) {
        int m = (i + j) / 2;
        if (a[m] > key) j = m;
        else i = m + 1;
    }
    return i
}

/* ====================================== */
/*  std::upper_bound(), std::lower_bound() */
/* ====================================== */
// search between [first, last)
// if no value is >= key (lb) / > key (ub), return last
int main () {
    vector<int> v{10,20,30,30,20,10,10,20};          // 10
20 30 30 20 10 10 20
    sort (v.begin(), v.end());                       // 10
10 10 20 20 20 30 30
    auto low = lower_bound (v.begin(), v.end(), 20); //
^
    auto up  = upper_bound (v.begin(), v.end(), 20); //
^
    cout << "lower_bound at position " << (low - v.begin())
<< '\n';
    cout << "upper_bound at position " << (up - v.begin())
<< '\n';
    return 0;
}

// ------------------------------------------------
// Query: how many items are LESS THAN (<) value x
lower_bound(v.begin(), v.end(), x) - v.begin();
// ------------------------------------------------
// Query: how many items are GREATER THAN (>) value x
v.end() - upper_bound(v.begin(), v.end(), x);
```

```cpp
//======================
// std::binary_search()
//======================
bool myfunction (int i,int j) { return (i<j); }
std::vector<int> v{1,2,3,4,5,4,3,2,1};
sort(v.begin(), v.end());
bool found = std::binary_search (v.begin(), v.end(), 6,
myfunction)

/* ======================== */
/* Discrete Ternary Search */
/* ======================== */
int cost(int i) { ... }
int min_search(int l, int r) { // convex cost function
    while (l < r) {
        int m = (i+j) >> 1;
        int slope = cost(m+1) - cost(m);
        if (slope >= 0) r = m;
        else l = m+1;
    }
    return l;
}
int max_search(int l, int r) { // concave cost function
    while (l < r) {
        int m = (i+j) >> 1;
        int slope = cost(m+1) - cost(m);
        if (slope <= 0) r = m;
        else l = m+1;
    }
    return l;
}
```

7.2 ternarysearch

```cpp
/* ======================== */
/* Ternary Search on Doubles */
/* ======================== */
double f(double x) { ... }
// find minimum (convex function)
double min_ternary_search(double l, double r, int times) {
    while (times--) {
        double d = (r - l) / 3.0;
        double m1 = l + d, m2 = r - d;
        double f1 = f(m1), f2 = f(m2);
        if (f1 <= f2) r = m2;
        if (f1 >= f2) l = m1;
    }
    return f((l+r)*.5);
}
// find maximum (concave function)
double max_ternary_search(double l, double r, int times) {
    while (times--) {
        double d = (r - l) / 3.0;
        double m1 = l + d, m2 = r - d;
        double f1 = f(m1), f2 = f(m2);
        if (f1 >= f2) r = m2;
        if (f1 <= f2) l = m1;
    }
    return f((l+r)*.5);
}

/* ========================== */
/* Ternary Search on Integers */
/* ========================== */
int f(int i) { ... }
int min_search(int l, int r) { // convex function
    while (l < r) {
        int m = (i+j) >> 1;
        if (f(m+1) >= f(m)) r = m;
        else l = m+1;
    }
    return l;
}
int max_search(int l, int r) { // concave function
    while (l < r) {
        int m = (i+j) >> 1;
        if (f(m+1) <= f(m)) r = m;
        else l = m+1;
    }
    return l;
}
```

8 Graphs

8.1 articulation points, cut edges, biconnected components

```cpp
// -------------------
// Tarjan's Algorithm
// -------------------
// references:
// https://www.youtube.com/watch?v=jFZsDDB0-vo
// https://www.hackerearth.com/practice/algorithms/graphs/
articulation-points-and-bridges/tutorial/
// https://www.hackerearth.com/practice/algorithms/graphs/
biconnected-components/tutorial/
// http://web.iitd.ac.in/~bspanda/biconnectedMTL776.pdf
vector<int> D, L;
vector<vector<int>> g;
stack<ii> s;

void print_and_remove_bicomp(int u, int v) {
    puts("biconnected component found:");
    ii uv(u,v);
    while (true) {
        ii top = s.top(); s.pop();
        printf("(%d, %d)\n", top.ff, top.ss);
```

```
        if (top == uv) break;
    }
}
/*
    // How to use:
    D.assign(g.size(), -1);
    L.resize(g.size());
    dfs(0, -1, 0);
 */
// general version: find everything
int root_children = 0;
void dfs(int u, int p, int d) { // (node, parent, depth)
    D[u] = L[u] = d; // u at least can reach itself
(ignoring u-p edge)
    for(int v : g[u]) {
        if (v == p) continue; // direct edge to parent ->
ignore
        if (D[v] == -1) { // exploring a new, unvisited
child node
            s.emplace(u,v); // add edge to stack
            dfs(v, u, d+1); // explore recursively v's
subtree
            // 1) detect articulation points and biconnected
components
            if (p == -1) { // 1.1) special case: if u is
root
                if (++root_children == 2) {
                    // we detected that root has AT LEAST 2
children
                    // therefore root is an articulation
point
                    printf("root = %d is articulation
point\n", root);
                }
                // whenever we come back to the root, we
just finished
                // exploring a whole biconnected component
                print_and_remove_bicomp(u,v);
            } else if (L[v] >= d) { // 1.2) general case:
non-root
                printf("u = %d is articulation point\n",
u);
                // we entered through and came back to an
AP,
                // so we just finished exploring a whole
biconnected component
                print_and_remove_bicomp(u,v);
            }
            // 2) detect cut edges (a.k.a. bridges)
            if (L[v] > D[u]) {
                printf("(u,v) = (%d, %d) is cut edge\n", u,
v);
            }
            // propagate low
            L[u] = min(L[u], L[v]);
        } else if (D[v] < d) { // back-edge to proper
ancestor
            s.emplace(u,v); // add edge to stack
            L[u] = min(L[u], D[v]); // propagate low
        } else { // forward-edge to an already visited
descendant
            // => do nothing, because this edge was already
considered as a
            // back-edge from v -> u
        }
    }
}

// find cut edges
void dfs(int u, int p, int d) {
    D[u] = L[u] = d;
    for (int v : g[u]) {
        if (v == p) continue;
        if (D[v] == -1) {
            dfs(v, u, d+1);
            if (L[v] > D[u]) printf("(u,v) = (%d, %d) is cut
edge\n", u, v);
            L[u] = min(L[u], L[v]);
        } else L[u] = min(L[u], D[v]); // back edge
    }
}

// find articulation points
int root_children = 0;
void dfs(int u, int p, int d) {
    D[u] = L[u] = d;
    for(int v : g[u]) {
        if (v == p) continue;
        if (D[v] == -1) {
            dfs(v, u, d+1);
            if (p == -1) { if (++root_children == 2)
printf("root = %d is AP\n", u); }
            else if (L[v] >= d) printf("u = %d is AP\n",
u);
            L[u] = min(L[u], L[v]);
        } else L[u] = min(L[u], D[v]);
    }
}

// find biconnected components
void dfs(int u, int p, int d) {
    D[u] = L[u] = d;
    for (int v : g[u]) {
        if (v == p) continue;
        if (D[v] == -1) {
            s.emplace(u,v);
            dfs(v, u, d+1);
            if (p == -1 or L[v] >= d)
print_and_remove_bicomp(u,v);
            L[u] = min(L[u], L[v]);
        } else if (D[v] < d) {
            s.emplace(u,v);
            L[u] = min(L[u], D[v]);
        }
    }
}
```

8.2 bellman ford

```
// complexity: O(|V||E|)
vector<int> bellman_ford(vector<vector<ii>>& g, int s, bool&
neg_cycle) {
    int n = g.size();
    vector<int> d(n, INT_MAX); // store min distance from s
to all nodes
    d[s] = 0;
    rep(_,1,n)  rep(u,0,n) if (d[u] < INT_MAX) for (auto&
e : g[u]) {
        d[e.ff] = min(d[e.ff], d[u] + e.ss);
    }
    // optional: detect negative cycle
    rep(u,0,n) if (d[u] < INT_MAX) for (auto& e : g[u]) {
        if (d[e.ff] > d[u] + e.ss) {
            neg_cycle = true; return d;
        }
    }
    return d;
}
```

8.3 diameter of tree

```
// ============================
// Find Tree's Diameter Ends
// ============================
const int MAXN = 10000;

int farthest_from(vector<vi>& g, int s) { // find farthest
node from 's' with BFS
    static int dist[MAXN];
    memset(dist, -1, sizeof(int) * g.size());
    int farthest = s;
    queue<int> q;
    q.push(s);
    dist[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
                if (dist[v] > dist[farthest]) farthest = v;
            }
        }
    }
    return farthest;
```

```
}

void find_diameter(vector<vi>& g, int& e1, int& e2) {
    e1 = farthest_from(g, 0);
    e2 = farthest_from(g, e1);
}
```

8.4 dijkstra

```
// complexity: (|E| + |V|) * log |V|
int N; // number of nodes
vector<vector<ii>> g; // graph
vector<int> D, P; // D: min distance from source to each
node
// P: parent of each node in shortest path from source
void dijkstra(int s) {
    P.assign(N, -1);
    D.assign(N, INT_MAX);
    D[s] = 0;
    priority_queue<ii, vector<ii>, greater<ii>> q; //
minheap
    q.emplace(0, s);
    while (q.size()) {
        int d, u; tie(d, u) = q.top(); q.pop(); // u = node,
d = mindist from s to u
        if (D[u] < d) continue; // skip outdated
improvements
        for (auto& e : g[u]) {
            if (D[e.ff] > d + e.ss) {
                D[e.ff] = d + e.ss;
                P[e.ff] = u;
                q.emplace(D[e.ff], e.ff);
            }
        }
    }
}
```

8.5 dinic

```
// Time Complexity:
// - general worst case: O (|E| * |V|^2)
// - unit capacities: O( min(V^(2/3), sqrt(E)) * E)
// - Bipartite graph (unit capacities) + source & sink (any
capacities): O(E sqrt V)
struct Dinic {
    struct Edge { int to, rev; ll f, c; };
    int n, t_; vector<vector<Edge>> G;
    vector<ll> D;
    vector<int> q, W;
    bool bfs(int s, int t) {
        W.assign(n, 0); D.assign(n, -1); D[s] = 0;
        int f = 0, l = 0; q[l++] = s;
        while (f < l) {
            int u = q[f++];
            for (const Edge &e : G[u]) if (D[e.to] == -1 &&
e.f < e.c)
                D[e.to] = D[u] + 1, q[l++] = e.to;
        }
        return D[t] != -1;
    }
    ll dfs(int u, ll f) {
        if (u == t_) return f;
        for (int &i = W[u]; i < (int)G[u].size(); ++i) {
            Edge &e = G[u][i]; int v = e.to;
            if (e.c <= e.f || D[v] != D[u] + 1) continue;
            ll df = dfs(v, min(f, e.c - e.f));
            if (df > 0) { e.f += df, G[v][e.rev].f -= df;
return df; }
        }
        return 0;
    }
    Dinic(int N) : n(N), G(N), D(N), q(N) {}
    void add_edge(int u, int v, ll cap) {
        G[u].push_back({v, (int)G[v].size(), 0, cap});
        G[v].push_back({u, (int)G[u].size() - 1, 0, 0}); //
Use cap instead of 0 if bidirectional
    }
    ll max_flow(int s, int t) {
        t_ = t; ll ans = 0;
        while (bfs(s, t)) while (ll dl = dfs(s, LLONG_MAX))
ans += dl;
        return ans;
    }
};

// usage
int main() {
    Dinic din(2);
    din.add_edge(0,1,10);
    ll mf = din.max_flow(0,1);
}
```

8.6 floyd warshall

```
// complexity: O(|V|^3)
#define rep(i,a,b) for(int i=a; i<b; ++i)
void floyd_warshall(vector<vector<int>>& g, bool& neg_cycle)
{
    int n = g.size();
    rep(k,0,n) rep(i,0,n) rep(j,0,n) {
        if (g[i][k] < INT_MAX and g[k][j] < INT_MAX) {
            g[i][j] = min(g[i][j], g[i][k] + g[k][j]);
        }
    }
    // optional: detect negative cycle
    rep(k,0,n) if (g[k][k] < 0) {
        rep(i,0,n) rep(j,0,n) if (g[i][k] < INT_MAX and g[k]
[j] < INT_MAX) {
            g[i][j] = -INT_MAX;
            neg_cycle = true;
        }
    }
}
```

8.7 level ancestor

```
/* ========================== */
/* LA (Level Ancestor Problem) */
/* ========================== */
const int MAXN = 10000;
const int MAXLOG = 16;
int P[MAXN][MAXLOG + 1]; // level ancestor table
int L[MAXN]; // level array
vector<vi> g; // tree graph
int root; // root of the tree

// dfs to record direct parents and levels
void dfs(int u, int p, int l) {
    P[u][0] = p;
    L[u] = l;
    for (int v : g[u])
        if (L[v] == -1)
            dfs(v, u, l + 1);
}

inline int log2(int x) { return 31 - __builtin_clz(x); }

void init_la(int n) {
    memset(P, -1, sizeof P);
    memset(L, -1, sizeof L);
    dfs(root, -1, 0);
    // fill sparse table
    int m = log2(n);
    rep(j, 1, m+1) {
        rep(i, 0, n) {
            // 2^j th ancestor of i
            // = 2^(j-1) th ancestor of 2^(j-1) th ancestor
of i
            int p = P[i][j-1];
            if (p != -1) P[i][j] = P[p][j-1];
        }
    }
}

int raise(int u, int steps) {
    // move node u "steps" levels up towards the root
    // i.e. find the steps-th ancestor of u
    for (int i = 0; steps; i++, steps>>=1) if (steps&1) u =
P[u][i];
    return u;
}
```

8.8 lowest common ancestor

```
//
------------------------------------------------------------
// METHOD 1: SPARSE TABLE - BINARY LIFTING (aka JUMP
POINTERS)
//
------------------------------------------------------------
// construction: O(|V| log |V|)
// query: O(log|V|)
// ** advantages:
//   - the lca query can be modified to compute querys over
the path between 2 nodes
//   - it's possible to append new leaf nodes to the tree
struct LCA {
    vector<int> A, D; // ancestors, depths
    vector<vector<int>> *g; // pointer to graph
    int n, maxe; // num nodes, max exponent
    int& anc(int u, int e) { return A[e * n + u]; }
    int inline log2(int x) { return 31 - __builtin_clz(x); }
    // dfs to record direct parents and depths
    void dfs(int u, int p, int depth) {
        anc(u,0) = p;
        D[u] = depth;
        for (int v : (*g)[u]) if (D[v] == -1) dfs(v, u,
depth + 1);
    }
    LCA(vector<vector<int>>& _g, int root) {
        g = &_g;
        n = _g.size();
        maxe = log2(n);
        D.assign(n, -1);
        A.resize(n * (maxe + 1));
        dfs(root, -1, 0);
        rep(e,1,maxe+1) {
            rep(u,0,n) {
                // u's 2^e th ancestor is
                // u's 2^(e-1) th ancestor's 2^(e-1) th
ancestor
                int a = anc(u,e-1);
                anc(u,e) = (a == -1 ? -1 : anc(a,e-1));
            }
        }
    }
    // move node u "k" levels up towards the root
    // i.e. find the k-th ancestor of u
    int raise(int u, int k) {
        for (int e = 0; k; e++, k>>=1) if (k&1) u =
anc(u,e);
        return u;
    }
    int lca(int u, int v) {
        if (D[u] < D[v]) swap(u, v);
        u = raise(u, D[u] - D[v]); // raise lowest to same
level
        if (u == v) return u; // same node, we are done
        // raise u and v to their highest ancestors below
the LCA
        invrep(e,maxe,0) {
            // greedily take the biggest 2^e jump possible
as long as
            // u and v still remain BELOW the LCA
            if (anc(u,e) != anc(v,e)) u = anc(u,e), v =
anc(v,e);
        }
        // the direct parent of u (or v) is lca(u,v)
        return anc(u,0);
    }
    // distance between 'u' and 'v'
    int dist(int u, int v) { // O(log(N))
        return D[u] + D[v] - 2 * D[lca(u,v)];
    }
    // optimized version (in case you already computed
lca(u,v))
    int dist(int u, int v, int lca_uv) { // O(1)
        return D[u] + D[v] - 2 * D[lca_uv];
    }
    // get the node located k steps from 'u' walking towards
'v'
    int kth_node_in_path(int u, int v, int k) {
        int lca_uv = lca(u,v);
        if (D[u] - D[lca_uv] >= k) return raise(u, k);
        return raise(v, dist(u,v,lca_uv) - k);
    }
    int add_child(int p, int u) { // optional
        // add to graph
        (*g)[p].push_back(u);
        // update depth
        D[u] = D[p] + 1;
        // update ancestors
        anc(u,0) = p;
        rep(e,1,maxe){
            p = anc(p,e-1);
            if (p == -1) break;
            anc(u,e) = p;
        }
    }
};

// ----------------------------------------
// METHOD 2: SPARSE TABLE - EULER TOUR + RMQ
// ----------------------------------------
// construction: O(2|V| log 2|V|) = O(|V| log |V|)
// query: O(1) (** assuming that __builtin_clz is mapped to
an
//               efficient processor instruction)
struct LCA {
    vector<int> E, D, H; // E = euler tour, D = depth, H =
first index of node in euler tour
    vector<int> DP; // memo for range minimun query
    vector<vector<int>> *g; // pointer to graph
    int idx; // tracks node ocurrences
    int n; // number of nodes
    int& rmq(int i, int e) { return DP[e * idx + i]; }
    inline int log2(int x) { return 31 - __builtin_clz(x); }
    void dfs(int u, int depth) {
        H[u] = idx; // index of first u's ocurrence
        E[idx] = u; // record node ocurrence
        D[idx++] = depth; // record depth
        for (int v : (*g)[u]) {
            if (H[v] == -1) {
                dfs(v, depth + 1); // explore v's subtree
and come back to u
                E[idx] = u; // new ocurrence of u
                D[idx++] = depth;
            }
        }
    }
    LCA(vector<vector<int>>& _g, int root) {
        g = &_g;
        n = _g.size();
        H.assign(n, -1);
        E.resize(2*n);
        D.resize(2*n);
        idx = 0;
        dfs(root, 0); // euler tour
        int nn = idx; // <-- make sure you use the correct
number
        int maxe = log2(nn);
        DP.resize(nn * (maxe+1));
        // build sparse table with bottom-up DP
        rep(i,0,nn) rmq(i,0) = i; // base case
        rep(e,1,maxe+1) { // general cases
            rep(i,0,nn-(1<<e)+1) {
                // i ... i + 2 ^ (e-1) - 1
                int i1 = rmq(i,e-1);
                // i + 2 ^ (e-1) ... i + 2 ^ e  - 1
                int i2 = rmq(i + (1 << (e-1)), e-1);
                // choose index with minimum depth
                rmq(i,e) = D[i1] < D[i2] ? i1 : i2;
            }
        }
    }
    int lca(int u, int v) {
        // get ocurrence indexes in increasing order
        int l = H[u], r = H[v];
        if (l > r) swap(l, r);
        // get node with minimum depth in range [l .. r] in
O(1)
        int len = r - l + 1;
        int e = log2(len);
        int i1 = rmq(l,e);
        int i2 = rmq(r - ((1 << e) - 1), e);
        return D[i1] < D[i2] ? E[i1] : E[i2];
    }
```

```
    int dist(int u, int v) {
        // make sure you use H to retrieve the indexes of u
and v
        // within the Euler Tour sequence before using D
        return D[H[u]] + D[H[v]] - 2 * D[H[lca(u,v)]];
    }
};

// -----------------
// EXAMPLE OF USAGE
// -----------------
int main() {
    // build graph
    int n, m; cin >> n >> m;
    vector<vector<int>> g(n);
    while (m--) {
        int u, v; cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    // init LCA
    LCA lca(g,0);
    // answer queries
    int q; cin >> q;
    while (q--) {
        int u, v; cin >> u >> v;
        cout << "LCA(" << u << "," << v << ") = " <<
lca.lca(u,v) << '\n';
        cout << "dist(" << u << "," << v << ") = " <<
lca.dist(u,v) << '\n';
    }
    return 0;
};
```

8.9 minimum spanning tree

```
/* ================= */
/* METHOD 1: KRUSKAL */
/* ================= */
struct Edge {
    int u, v, cost;
    bool operator<(const Edge& o) const { return cost <
o.cost; }
};
namespace Kruskal {
    struct UnionFind {
        vector<int> p, rank;
        UnionFind(int n) {
            rank.assign(n,0);
            p.resize(n);
            rep(i,0,n) p[i] = i;
        }
        int findSet(int i) { return (p[i] == i) ? i : (p[i]
= findSet(p[i])); }
        bool isSameSet(int i, int j) { return findSet(i) ==
findSet(j); }
        void unionSet(int i, int j) {
            if (!isSameSet(i, j)) {
                int x = findSet(i), y = findSet(j);
                if (rank[x] > rank[y]) { p[y] = x; }
                else { p[x] = y; if (rank[x] == rank[y])
rank[y]++; }
            }
        }
    };
    int find_mst(int n_nodes, vector<Edge>& edges,
vector<vector<ii>>& mst) {
        sort(edges.begin(), edges.end());
        UnionFind uf(n_nodes);
        mst.assign(n_nodes, vector<ii>());
        int mstcost = 0;
        int count = 1;
        for (auto& e : edges) {
            int u = e.u, v = e.v, cost = e.cost;
            if (!uf.isSameSet(u, v)) {
                mstcost += cost;
                uf.unionSet(u, v);
                mst[u].emplace_back(v, cost);
                mst[v].emplace_back(u, cost);
                if (++count == n_nodes) break;
            }
        }
        return mstcost;
    }
}
/* ============== */
/* METHOD 2: PRIM */
/* ============== */
struct Edge {
    int u, v, cost;
    bool operator<(const Edge& o) const { return cost >
o.cost; } // notice > instead of <
};
namespace Prim {
    bool visited[MAXN];
    int find_mst(vector<vector<ii>>& g, vector<vector<ii>>&
mst) {
        int n_nodes = g.size();
        memset(visited, false, sizeof(bool) * n_nodes);
        mst.assign(n_nodes, vector<ii>());
        priority_queue<Edge> q;
        int total_cost = 0;
        visited[0] = true;
        for (ii& p : g[0]) q.push({0, p.ff, p.ss});
        int count = 1;
        while (!q.empty()) {
            Edge e = q.top(); q.pop();
            int u = e.u, v = e.v, cost = e.cost;
            if (visited[v]) continue;
            visited[v] = true;
            total_cost += cost;
            mst[u].emplace_back(v, cost);
            mst[v].emplace_back(u, cost);
            if (++count == n_nodes) break;
            for (ii& p : g[v]) {
                if (visited[p.ff]) continue;
                q.push({v, p.ff, p.ss});
            }
        }
        return total_cost;
    }
}
```

8.10 strongly connected components

```
// Tarjan's SCC algorithm
vector<int> s, D, L;
vector<bool> instack;
vector<vector<int>>* g;
int n, ID;
void dfs(int u) {
    D[u] = L[u] = ID++;
    instack[u] = true;
    s.push_back(u);
    for (int v : (*g)[u]) {
        if (D[v] == -1) {
            dfs(v);
            L[u] = min(L[v], L[u]);
        } else if (instack[v]) {
            L[u] = min(L[v], L[u]);
        }
    }
    if (L[u] == D[u]) { // u is root of a SCC
        cout << "SCC found!\n";
        while (1) { // remove SCC from top of the stack
            int x = s.back(); s.pop_back();
            instack[x] = false;
            if (x == u) break;
        }
    }
}
void find_sccs(vector<vector<int>>& _g) {
    n = _g.size(); ID = 0;
    s.reserve(n); D.assign(n, -1); L.resize(n);
    instack.assign(n, 0);
    g = &_g;
    rep(u,0,n) if (D[u] == -1) dfs(u);
}
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> g(n);
    while(m--) {
        int u, v; cin >> u >> v; u--, v--;
        g[u].push_back(v);
    }
```

```
    find_sccs(g);
    return 0;
}
```

8.11 toposort

```
// ---------------------------
// option 1: tarjan's algorithm
// ---------------------------
// Note: nodes are sorted in reversed order
vector<vector<int>> g; // graph
int n; // num of nodes
bool vis[MAXN]; // track visited nodes
vector<int> sorted;
void dfs(int u) {
    vis[u] = true;
    for (int v : g[u]) if (!vis[v]) dfs(v);
    sorted.push_back(u);
}
void topo_sort() {
    memset(vis, false, sizeof(bool) * n);
    sorted.clear();
    rep(i,0,n) if (!vis[i]) dfs(i);
}

// ---------------------------
// option 2: Kahn's algorithm
// ---------------------------
vector<int> topoSort(vector<vector<int>>& g) {
    int n = g.size();
    vector<int> indegree(n,0);
    rep(u,0,n) for (int v : g[u]) indegree[v]++;
    queue<int> q;
    int count = 0;
    rep(u,0,n) if (indegree[u] == 0) q.push(u);
    vector<int> sorted;
    sorted.reserve(n);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        sorted.push_back(u);
        for (int v : g[u]) if (--indegree[v] == 0)
q.push(v);
    }
    return sorted;
}
// special variant: check if a directed graph is a DAG
bool isDAG(vector<vector<int>>& g) {
    int n = g.size();
    vector<int> indegree(n,0);
    rep(u,0,n) for (int v : g[u]) indegree[v]++;
    queue<int> q;
    int count = 0;
    rep(u,0,n) if (indegree[u] == 0) q.push(u);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        count++;
        for (int v : g[u]) if (--indegree[v] == 0)
q.push(v);
    }
    return count == n;
}
```

9 Geometry

9.1 2D / point2d

```
const double PI = acos(-1);
const double EPS = 1e-12;
typedef double T;
struct P { // 2D
    T x, y;
    P() {}
    P(T x, T y) : x(x), y(y) {}
    bool operator==(const P& p) const { return x==p.x && y
== p.y; }
    P operator+(const P& p) const { return {x+p.x, y+p.y}; }
    P operator-(const P& p) const { return {x-p.x, y-p.y}; }
    P operator*(T d) const { return {x*d, y*d}; }
    P operator/(T d) const { return {x/d, y/d}; }
    T operator^(const P& p) const { return x*p.y -
y*p.x; } // cross product
    T operator*(const P& p) const { return x*p.x +
y*p.y; } // dot product
    P neg() { return { -x, -y }; }
    P rot_ccw() { return { -y, x }; }
    P rot_cw() { return { y, -x }; }
    T norm2() { return x*x + y*y; }
    double norm() { return sqrt(norm2()); }
    int quad() const { return (x >= 0) ? (y >= 0 ? 0: 3) :
(y >= 0 ? 1 : 2); } // 0, 1, 2, 3
    double angle() { return atan2(y, x); } // [-PI, PI]
    P unit() { return (*this)/norm(); }
    bool operator<(const P& p) const { // smaller quadrant
or same quadrant and to the right
        int q = quad(), qp = p.quad();
        return q != qp ? q < qp : ((*this)^p) > 0;
    }
    bool same_angle(const P& p) { // two vectors pointing to
the same direction
        return quad() == p.quad() and ((*this)^p) == 0;
    }
};
```

9.2 2D / common 2d algorithms

```
T turn(P &a, P &b, P &c) { return (b-a)^(c-a); }

int turn_sign(P& a, P& b, P& c) { T t = turn(a,b,c); return
t < 0 ? -1 : t == 0 ? 0 : 1; /* (t < -EPS) ? -1 : (t >
EPS) ? 1 : 0; */ }

bool on_line(P& a, P& b, P& p) { return turn(a,b,p) == 0; /*
abs(turn(a,b,p)) < EPS; */}

bool in_disk(P &a, P &b, P &p) { return (a-p)*(b-p) <= 0; /*
(a-p)*(b-p) <= EPS; */ }

bool on_segment(P &a, P &b, P &p) { return on_line(a,b,p) &&
in_disk(a,b,p); }

double closest_t(P& a, P& b, P& p) { return -((a-p)*(b-a) /
((b-a)*(b-a)*2.)); }

/* ========================================================
*/
/* Check if a segment is below another segment (wrt a ray)
*/
/* ========================================================
*/
// i.e: check if a segment is intersected by the ray first
// Assumptions:
// 1) for each segment:
//  p1 should be LEFT (or COLLINEAR) and p2 should be RIGHT
(or COLLINEAR) wrt ray
// 2) segments do not intersect each other
// 3) segments are not collinear to the ray
// 4) the ray intersects all segments
struct Segment { P p1, p2;};
Segment segments[100000]; // array of line segments
bool is_si_below_sj(int i, int j) { // custom comparator
based on cross product
    Segment& si = segments[i];
    Segment& sj = segments[j];
    return (si.p1.x >= sj.p1.x) ?
        turn(si.p1, sj.p2, sj.p1) > 0:
        turn(sj.p1, si.p1, si.p2) > 0;
}
// this can be used to keep a set of segments ordered by
order of intersection
// by the ray, for example, active segments during a SWEEP
LINE
set<int, decltype(is_si_below_sj)>
active_segments(is_si_below_sj); // ordered set


// Rectangle Intersection
bool do_rects_intersect(P& dl1, P& ur1, P& dl2, P& ur2) {
    return max(dl1.x, dl2.x) <= min(ur1.x, ur2.x) &&
max(dl1.y, dl2.y) <= min(ur1.y, ur2.y);
}

// Line Segment Intersection
bool do_segments_intersect(P& p1, P& q1, P& p2, P& q2) {
    int t11 = turn_sign(p1, q1, p2), t12 = turn_sign(p1, q1,
```

```
q2);
    int t21 = turn_sign(p2, q2, p1), t22 = turn_sign(p2, q2,
q1);
    if (t11 != t12 and t21 != t22) return true;
    if (t11 == t12 and t11 == 0) return on_segment(p1,q1,p2)
or on_segment(p2,q2,p1);
    return false;
}

// Line - Line Intersection
// return whether straight lines <-a-b-> and <-c-d->
intersect each other at a specific point
// if they do, we assign values to t1 and t2 such that
//    a + (b - a) * t1 == c + (d - c) * t2
bool intersect_lines(const P& a, const P& b, const P& c,
const P& d, double& t1, double& t2) {
    P c_d = c-d;
    P b_a = b-a;
    double x = b_a^c_d;
    if (x == 0) return false; /* if (abs(x) < EPS) */ //
parallel
    P c_a = c-a;
    t1 = c_a^c_d/x;
    t2 = b_a^c_a/x;
    return true;
}

// Circle Intersection
struct Circle {
    P c;
    double r;
    Circle(P c, double r) : c(c), r(r) {}
    Circle(double x, double y, double r) : c(P(x, y)), r(r)
{}
};
bool fully_outside(double r1, double r2, double d_sqr) {
    double tmp = r1 + r2; return d_sqr > tmp * tmp;
}
bool fully_inside(double r1, double r2, double d_sqr) {
    if (r1 > r2) return false;
    double tmp = r2 - r1; return d_sqr < tmp * tmp;
}
bool do_circles_intersect(Circle& c1, Circle& c2) {
    double dx = c1.x - c2.x, dy = c1.y - c2.y;
    double d_sqr = dx * dx + dy * dy;
    if (fully_inside(c1.r, c2.r, d_sqr)) return false;
    if (fully_inside(c2.r, c1.r, d_sqr)) return false;
    if (fully_outside(c1.r, c2.r, d_sqr)) return false;
    return true;
}

// Based on https://cp-algorithms.com/geometry/circle-line-
intersection.html
// Circle - Line Intersection
// Returns whether a circle and a line intersect each other.
// There are 3 possible cases:
// 1) The line is tangent to the circle (one intersection
point)
// 2) The line intersects the circle at two points
// 3) The line does not intersect the circle
// If there are 2 intersection points, we assign values to
ip1 and ip2.
// If there is 1 intersection point, we assign values to
ip1.
// If there are no intersection points, we return false.
struct Line {
    P p1, p2;
    Line(P p1, P p2) : p1(p1), p2(p2) {}
    Line(double x1, double y1, double x2, double y2) :
p1(P(x1, y1)), p2(P(x2, y2)) {}
};
bool intersect_circle_line(Circle& c, Line& l, int& n, P&
ip1, P& ip2) {
    // Calculate coefficients a, b, c for the line equation
a*x + b*y + c = 0,
    // given the circle center (c.c.x, c.c.y), radius c.r,
and the line l (l.p1, l.p2).

    // Step 1: Calculate line points relative to the circle
center
    P p1 = l.p1 - c.c;
    P p2 = l.p2 - c.c;

    // Step 2: Calculate line co    efficients
    double a = p2.y - p1.y;
    double b = p1.x - p2.x;
    double cc = -a * p1.x - b * p1.y;

    // Step 3: Calculate the projected point (x0, y0):
closest point on the line to the center (now (0, 0))
    double x0 = -a * cc / (a * a + b * b);
    double y0 = -b * cc / (a * a + b * b);

    // Step 4: Check number of intersection points by
comparing squared distance from center to line
    // with the squared radius
    double sq = a * a + b * b;
    if (cc * cc > c.r * c.r * sq + EPS) {
        // Case 1: No intersection
        n = 0;
        return false;
    } else if (std::abs(cc * cc - c.r * c.r * sq) < EPS) {
        // Case 2: Tangent (one intersection)
        ip1 = P(x0 + c.c.x, y0 + c.c.y); // transform back
to original coordinates
        n = 1;
        return true;
    } else {
        // Case 3: Two intersection points
        double d = c.r * c.r - cc * cc / sq;
        double mult = sqrt(d / sq);

        ip1 = P(x0 + b * mult + c.c.x, y0 - a * mult +
c.c.y);
        ip2 = P(x0 - b * mult + c.c.x, y0 + a * mult +
c.c.y);
        n = 2;
        return true;
    }
}

// Point - Line / Line Segment distance
// reference: https://stackoverflow.com/questions/849211/
shortest-distance-between-a-point-and-a-line-segment
// get distance between p and projection of p on line <- a -
b ->
double point_line_dist(P& p, P& a, P& b) {
    P d = b-a;
    double t = (d*(p-a)) / d.norm2();
    return (a + d * t - p).norm();
}
// get distance between p and truncated projection of p on
segment a -> b
double point_segment_dist(P& p, P& a, P& b) {
    if (a==b) return (p-a).norm(); // segment is a single
point
    P d = b-a; // direction
    double t = (d*(p-a)) / d.norm2();
    if (t <= 0) return (p-a).norm(); // truncate left
    if (t >= 1) return (p-b).norm(); // truncate right
    return (a + d * t - p).norm();
}

// Straight Line Hashing (integer coords)
// ----------------------------------------
// task: given 2 points p1, p2 with integer coordinates,
output a unique
// representation {a,b,c} such that a*x + b*y + c = 0 is the
equation
// of the straight line defined by p1, p2. This
representation must be
// unique for each straight line, no matter which p1 and p2
are sampled.
tuple<ll,ll,ll> hash_line(const P& p1, const P& p2) {
    ll a = p1.y - p2.y;
    ll b = p2.x - p1.x;
    ll c = p1.x * (p2.y - p1.y) - p1.y * (p2.x - p1.x);
    ll sgn = (a < 0 or (a == 0 and b < 0)) ? -1 : 1;
    ll g = __gcd(abs(a), __gcd(abs(b), abs(c))) * sgn;
    return make_tuple(a/g, b/g, c/g);
}
// task: given 2 points p1 and p2 with integer coords,
return a pair {a, b}
// which is unique for all straight lines having the same
slope as
```

```cpp
// the straight line that goes through p1 and p2
pair<ll,ll> hash_slope(const P& p1, const P& p2) {
    ll dx = p2.x - p1.x;
    ll dy = p2.y - p1.y;
    ll sgn = (dx < 0 or (dx == 0 and dy < 0)) ? -1 : 1;
    ll g = __gcd(abs(dx), abs(dy)) * sgn;
    return {dx/g, dy/g};
}

// Circumcenter of 3 points
// reference: https://codeforces.com/blog/entry/22194
P bary(const P& A, const P& B, const P& C, double a, double
b, double c) {
    return (A*a + B*b + C*c) / (a + b + c);
}
P circumcenter(const P& A, const P& B, const P& C) {
    double a = (B - C).norm2(), b = (C - A).norm2(), c = (A
- B).norm2();
    return bary(A, B, C, a*(b+c-a), b*(c+a-b), c*(a+b-c));
}

// Area of 2D non self intersecting Polygon
// based on Green's Theorem:
// http://math.blogoverflow.com/2014/06/04/greens-theorem-
and-area-of-polygons/
double polygon_area(vector<P>& p) { // ** points must be
sorted ccw or cw
    int n = p.size(); double a = 0;
    rep(i,0,n) { int j = (i+1) % n; a += (p[i].x + p[j].x) *
(p[j].y - p[i].y); }
    return a * 0.5; // abs(a * 0.5) if points are cw
}

// Point in Polygon
// ----------------------------
// 1) Convex Polygons
// 1.1) O(N) method
bool in_convex_poly(P& p, vector<P>& cp) {
    int n = cp.size();
    rep(i,0,n) if (turn(cp[i], cp[(i+1)%n], p) < 0) return
false;
    return true;
}
// 1.2) O(log N) method
bool in_triangle(P& a, P& b, P& c, P& x) {
    return turn(a, b, x) >= 0 and turn(b, c, x) >= 0 and
turn(c, a, x) >= 0;
}
bool in_convexhull(P& p, vector<P>& ch) {
    if (turn(ch[0], ch[1], p) < 0) return false;
    if (turn(ch[0], ch.back(), p) > 0) return false;
    int l = 2, r = ch.size() - 1;
    while (l < r) {
        int m = (l+r) >> 1;
        if (turn(ch[0], ch[m], p) <= 0) r = m;
        else l = m+1;
    }
    return in_triangle(ch[0], ch[l-1], ch[l], p);
}
// ----------------------------------------------
// 2) General methods: for complex / simple polygons
// EvenOdd Rule (ray casting - crossing number)
bool above(P &a, P &p) { return p.y >= a.y; }
bool crosses_ray(P &a, P &p, P &q) {
    return (above(a, q) - above(a, p)) * turn(a, p, q) > 0;
}
bool in_polygon_evenodd(vector<P> &p, P a, bool strict =
true) {
    int c = 0, n = p.size();
    rep(i,0,n) {
        int j = (i+1)%n; if (on_segment(p[i], p[j], a))
return !strict;
        c += crosses_ray(a, p[i], p[j]);
    }
    return c & 1;
}
// Nonzero Rule (winding number)
bool in_polygon_nonzero(vector<P> &pol, P p, bool strict =
true) {
    int wn = 0, n = pol.size();
    rep(i,0,n) {
        int j = (i+1)%n; if (on_segment(pol[i], pol[j], p))
return !strict;
        P &a = pol[i], &b = pol[j];
        if (a.y <= p.y) {
            if (p.y < b.y and ((b-a)^(p-a)) > 0) ++wn; //
upward & left
        } else {
            if (p.y >= b.y and ((b-a)^(p-a)) < 0) --wn; //
downward & right
        }
    }
    return wn != 0; // nonzero
}
```

9.3 closest pair of points

```cpp
// Taken from https://cp-algorithms.com/geometry/nearest_
points.html
using ll = long long;
using ld = long double;
struct pt {
    ll x, y;
    pt() {}
    pt(ll x_, ll y_) : x(x_), y(y_) {}
    void read() { cin >> x >> y; }
};
bool operator==(const pt& a, const pt& b) {
    return a.x == b.x and a.y == b.y;
}
struct CustomHashPoint {
    size_t operator()(const pt& p) const {
        static const uint64_t C =
chrono::steady_clock::now().time_since_epoch().count();
        return C ^ ((p.x << 32) ^ p.y);
    }
};
ll dist2(pt a, pt b) {
    ll dx = a.x - b.x, dy = a.y - b.y;
    return dx*dx + dy*dy;
}
pair<int,int> closest_pair_of_points(vector<pt> P) {
    int n = int(P.size());
    assert(n >= 2);
    // if there is a duplicated point, we have the solution
    unordered_map<pt,int,CustomHashPoint> previous;
    for (int i = 0; i < int(P.size()); ++i) {
        auto it = previous.find(P[i]);
        if (it != previous.end()) {
            return {it->second, i};
        }
        previous[P[i]] = i;
    }
    unordered_map<pt,vector<int>,CustomHashPoint> grid;
    grid.reserve(n);
    mt19937
rd(chrono::system_clock::now().time_since_epoch().count());
    uniform_int_distribution<int> dis(0, n-1);
    ll d2 = dist2(P[0], P[1]);
    pair<int,int> closest = {0, 1};
    auto candidate_closest = [&](int i, int j) -> void {
        ll ab2 = dist2(P[i], P[j]);
        if (ab2 < d2) {
            d2 = ab2;
            closest = {i, j};
        }
    };
    for (int i = 0; i < n; ++i) {
        int j = dis(rd);
        int k = dis(rd);
        while (j == k) k = dis(rd);
        candidate_closest(j, k);
    }
    ll d = ll( sqrt(ld(d2)) + 1 );
    for (int i = 0; i < n; ++i) {
        grid[{P[i].x/d, P[i].y/d}].push_back(i);
    }
    // same block
    for (const auto& it : grid) {
        int k = int(it.second.size());
        for (int i = 0; i < k; ++i) {
            for (int j = i+1; j < k; ++j) {
                candidate_closest(it.second[i],
it.second[j]);
            }
```

```
        }
    }
    // adjacent blocks
    for (const auto& it : grid) {
        auto coord = it.first;
        for (int dx = 0; dx <= 1; ++dx) {
            for (int dy = -1; dy <= 1; ++dy) {
                if (dx == 0 and dy == 0) continue;
                pt neighbour = pt(
                    coord.x  + dx,
                    coord.y + dy
                );
                for (int i : it.second) {
                    if (not grid.count(neighbour)) continue;
                    for (int j : grid.at(neighbour)) {
                        candidate_closest(i, j);
                    }
                }
            }
        }
    }
    return closest;
}
```

9.4 convex hull

```
// ------------------------------------------------
// Convex Hull: Andrew's Montone Chain Algorithm
// ------------------------------------------------
struct Point {
    ll x, y;
    ll operator^(const Point<ll>& p) const { return x*p.y -
y*p.x; }
    bool operator<(const Point& p) const {
        return x < p.x || (x == p.x && y < p.y);
    }
};
ll turn(Point& a, Point& b, Point& c) { return (b-a)^(c-
a); }

vector<Point> upper_hull(vector<Point>& P) {
    // sort points lexicographically
    int n = P.size(), k = 0;
    sort(P.begin(), P.end());
    // build upper hull
    vector<Point> uh(n);
    invrep(i,n-1,0) {
        while (k >= 2 && turn(uh[k-2], uh[k-1], P[i]) <= 0)
k--;
        uh[k++] = P[i];
    }
    uh.resize(k);
    return uh;
}
```

```
vector<Point> lower_hull(vector<Point>& P) {
    // sort points lexicographically
    int n = P.size(), k = 0;
    sort(P.begin(), P.end());
    // collect lower hull
    vector<Point> lh(n);
    rep(i,0,n) {
        while (k >= 2 && turn(lh[k-2], lh[k-1], P[i]) <= 0)
k--;
        lh[k++] = P[i];
    }
    lh.resize(k);
    return lh;
}

vector<Point> convex_hull(vector<Point>& P) {
    int n = P.size(), k = 0;
    // set initial capacity
    vector<Point> H(2*n);
    // sort points lexicographically
    sort(P.begin(), P.end());
    // build lower hull
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && turn(H[k-2], H[k-1], P[i]) <= 0)
k--;
        H[k++] = P[i];
    }
    // build upper hull
    for (int i = n-2, t = k+1; i >= 0; i--) {
        while (k >= t && turn(H[k-2], H[k-1], P[i]) <= 0)
k--;
        H[k++] = P[i];
    }
    // remove extra space
    H.resize(k-1);
    return H;
}
```

9.5 greens theorem

```
struct Point { double x, y; };
// Computes the line integral of the vector field <0,x> over
the arc of the circle with radius 'r'
// and x-coordinate 'x' from angle 'a' to angle 'b'. The 'y'
goes away in the integral so it
// it doesn't matter.
// This can be done using a parameterization of the arc in
polar coordinates:
// x(t) = x + r * cos(t)
// y(t) = y + r * sin(t)
// a <= t <= b
// The final integral can be seen here:
// https://www.wolframalpha.com/input/?i=integral((x+%2B+r*
cos(t))+*+derivative(y+%2B+r*sin(t))+*+dt,+t%3Da..b)
double arc_integral(double x, double r, double a, double b)
{
    return x * r * (sin(b) - sin(a)) + r * r * 0.5 * (0.5 *
(sin(2*b) - sin(2*a)) + b - a);
}
// Computes the line integral of the vector field <0, x>
over the directed segment a -> b
// This can be done using the parameterization:
// x(t) = a.x + (b.x - a.x) * t
// y(t) = a.y + (b.y - a.y) * t
// 0 <= t <= 1
double segment_integral(Point& a, Point& b) { return 0.5 *
(a.x + b.x) * (b.y - a.y); }
```

9.6 smallest enclosing circle

```
// Find the smallest circle enclosing a list of points using
Welzl's randomized algorithm
// O(N) expected time
// references: https://www.cse.iitk.ac.in/users/ssahai/
talks/sec.pdf
// https://www.nayuki.io/res/smallest-enclosing-circle/
computational-geometry-lecture-6.pdf
void smallest_enclosing_circle(vector<Point>& pts, Point&
center, double& r) {
    random_shuffle(pts.begin(), pts.end());
    center = pts[0]; r = 0;
    int N = pts.size();
    rep(i,1,N) {
        if ((pts[i] - center).norm() > r + EPS) {
            center = pts[i];
            r = 0;
            rep(j,0,i) {
                if ((pts[j] - center).norm() > r + EPS) {
                    center = (pts[i] + pts[j]) * 0.5;
                    r = (pts[i] - center).norm();
                    rep(k,0,j) {
                        if ((pts[k] - center).norm() > r +
EPS) {
                            center = circumcenter(pts[i],
pts[j], pts[k]);
                            r = (pts[k] - center).norm();
                        }
                    }
                }
            }
        }
    }
}
```

9.7 trigonometry

```
/* ================= */
/* Angle of a vector */
/* ================= */
const double PI = acos(-1);
```

```cpp
const double _2PI = 2 * PI;

double correct_angle(double angle) { // to ensure 0 <= angle
<= 2PI
    while (angle < 0) angle += _2PI;
    while (angle > _2PI) angle -= _2PI;
    return angle;
}
double angle(double x, double y) {
    // atan2 by itself returns an angle in range [-PI, PI]
    // no need to "correct it" if that range is ok for you
    return correct_angle(atan2(y, x));
}

/* ============== */
/* Cosine Theorem */
/* ============== */
// Given triangle with sides a, b and c, returns the angle
opposed to side a.
// a^2 = b^2 + c^2 - 2*b*c*cos(alpha)
// => alpha = acos((b^2 + c^2 - a^2) /(2*b*c))
double cos_theorem_angle(double a, double b, double c) {
    return acos((b*b + c*c - a*a)/(2*b*c));
}
```

10 Apuntes Javi

10.1 *template

```cpp
#pragma GCC optimize("Ofast")
#include<bits/stdc++.h>
using namespace std;
//la verdadera clave del éxito
#define int long long

typedef long long ll;
typedef pair<int,int> pii;
typedef pair<ll,ll> pll;
typedef string str;
typedef vector<str> vs;
typedef vector<int> vi;
typedef vector<vi> vvi;
typedef vector<ll> vl;
typedef vector<vl> vvl;
typedef vector<pii> vpi;
typedef vector<vpi> vvpi;
typedef vector<str> vs;
typedef tuple<int, int, int> tiii;

#define fi first
#define se second
#define rep(i, a, b) for (int i = (int)a; i < (int)b; i++)
#define repr(i, a, b) for(int i = (int)a; i >= (int)b; i--)
#define pb push_back
#define mk make_pair
#define printv(v) for(auto x: v){cout<<x<<" ";}cout<<endl
#define printm(v) for(auto w: v){printv(w);}cout<<endl
#define deb1(s) cout<<s<<endl
#define deb2(s, s2) cout<<s<<" "<<s2<<endl
#define deb3(s, s2, s3) cout<<s<<" "<<s2<<" "<<s3<<endl
#define LINE deb1("====================")
#define sz(x) (int)(x).size()
#define all(x) begin(x), end(x)
#define allr(x) (x).rbegin(), (x).rend()

const int WSP = 998244353;
const int INF = 4e18;
const int MOD = 1e9 + 7;

void probar_punto() {}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(NULL);
    int puntos_a_probar = 1;
    //cin >> puntos_a_probar;
    while (puntos_a_probar--) { probar_punto(); }
    return 0;
    cout << "haiiiii ^_^ hi!! hiiiiii <3 haiiiiii hii :3" <<
endl;
}
```

10.2 conv binpow

```cpp
// usar con template de ntt

vec<ll> conv_binpow(vec<ll> a, ll b) {
    vec<ll> res = {1};
    while (b > 0) {
        if (b & 1)
            res = conv(res, a);
        a = conv(a, a);
        b >>= 1;
    }
    return res;
}
```

10.3 directed mst

```cpp
struct node {
  pair<ll, int> val;
  ll lazy;
  node *l, *r;
  node() {}
  node(pair<int, int> v) : val(v), lazy(0), l(NULL), r(NULL)
{}

  void prop() {
    val.first += lazy;
    if (l) l->lazy += lazy;
    if (r) r->lazy += lazy;
    lazy = 0;
  }
};
void merge(node*& a, node* b) {
  if (!a) swap(a, b);
  if (!b) return;
  a->prop(), b->prop();
  if (a->val > b->val) swap(a, b);
  merge(rand()%2 ? a->l : a->r, b);
}
pair<ll, int> pop(node*& R) {
  R->prop();
  auto ret = R->val;
  node* tmp = R;
  merge(R->l, R->r);
  R = R->l;
  if (R) R->lazy -= ret.first;
  delete tmp;
  return ret;
}
void apaga(node* R) { if (R) apaga(R->l), apaga(R->r),
delete R; }

ll dmst(int n, int r, vector<pair<pair<int, int>, int>>& ar)
{
  vector<int> p(n); iota(p.begin(), p.end(), 0);
  function<int(int)> find = [&](int k) { return p[k]==k?
k:p[k]=find(p[k]); };
  vector<node*> h(n);
  for (auto e : ar) merge(h[e.first.second], new
node({e.second, e.first.first}));
  vector<int> pai(n, -1), path(n);
  pai[r] = r;
  ll ans = 0;

  for (int i = 0; i < n; i++) { // vai conectando todo mundo
    int u = i, at = 0;
    while (pai[u] == -1) {
      if (!h[u]) { // nao tem
        for (auto i : h) apaga(i);
        return LINF;
      }
      path[at++] = u, pai[u] = i;
      auto [mi, v] = pop(h[u]);
      ans += mi;

      if (pai[u = find(v)] == i) { // ciclo
        while (find(v = path[--at]) != u)
          merge(h[u], h[v]), h[v] = NULL, p[find(v)] = u;
        pai[u] = -1;
      }
    }
  }
  for (auto i : h) apaga(i);
```

```
  return ans;
}
```

10.4 fft de dudosa procedencia

```
int n;

const double PI = acos(-1);

typedef complex<double> C;
typedef vector<double> vd;

void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-
rolled)  /// include-line
      auto x = (double *)&rt[j+k], y = (double
*)&a[i+j+k];        /// exclude-line
      C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] +
x[1]*y[0]);           /// exclude-line
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}
const double EPS = 1e-9;
vl conv(const vl& a, const vl& b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = (ll)round(imag(out[i]) / (4 *
n));

  while(res.back()<EPS and sz(res)>1)res.pop_back();
  return res;
}
```

10.5 ntt

```
/* -
name = "NTT"
source = "https://github.com/miguelsotocarlos/cp-fft"
[info]
time = "$O(n log n)$"
- */
#include <bits/stdc++.h>
using namespace std;

template <typename T>
using vec = vector<T>;
using ll = long long;

#define sz(x) (int)(x).size()
#define rep(i, a, b) for(int i = a; i < (b); ++i)

// el largo maximo del vector resultante de la convolucion
es 2^LG
const int LG = 23;
template<class u, class uu, u p, u root>
struct FFT {
  u r[1+(2<<LG)];
  constexpr u m(u a, u b) {
    uu k = uu(a)*b;
    #define op(g) g*(g*p+2)
    k += u(k) * (op(op(op(op(op(-p))))) * uu(p);
    #undef op
    return u(k>>(8*sizeof(u)));
  }
  constexpr u red(u k, u a) { return a-k*(a>=k); }
  FFT() {
    u k = r[2<<LG] = -p%p, b=root, e = p>>LG;
    for(; e; e/=2, b=m(b,b)) if(e%2) k=m(k, b);
    for(int i = (2<<LG) - 1; i >= 0; i--) r[i]=red(p,
m(r[i+1], k)), i&(i-1)?0:k=m(k,k);
    assert(r[2] != r[3]); assert(r[1] == r[2]);
  }
  vec<ll> cv(const vec<ll> &as, const vec<ll> &bs, u *v) {
    int c=max(sz(as)+sz(bs)-1, 0), n=1;
    assert(c <= (1<<LG));
    u h=u(uu(-p)*-p%p), a=m(h, p/2+1), x, y;
    while(n<c) n*=2, h=red(p, m(h, a));
    rep(i, 0, n)
      v[i] = i<sz(as) ? u(as[i]) : 0,
      v[i+n] = i<sz(bs) ? u(bs[i]) : 0;
    for(auto s:{v,v+n})
    for(int j = n; j >= 2; j--) for(int k=j&-j; k/=2;)
rep(i, j-k, j)
      x=s[i], y=s[i-k],
      s[i-k] = red(2*p, x+y),
      s[i] = m(2*p+y-x, r[3*k-j+i]);
    rep(i, 0, n) v[i] = m(v[i], v[i+n]);
    rep(j, 2, n+1) for(int k=1; !(k&j); k*=2) rep(i, j-k, j)
      x = m(v[i], r[3*k+j-i]),
      y = red(2*p, v[i-k]),
      v[i-k]=x+y, v[i]=2*p+y-x;
    rep(i, 0, c) v[i] = red(p, m(v[i], h));
    return vec<ll>(v, v+c);
  }
};
// para convoluciones modulo 998244353:
vec<ll> conv(const vec<ll> &as, const vec<ll> &bs) {
  static uint32_t v[2<<LG];
  static FFT<uint32_t, uint64_t, 998244353, 3> fft;
  return fft.cv(as, bs, v);
}
```

10.6 segment tree

```
/*
 *Description:* Range queries, build $O(n)$, query and
update $O(log n)$, positions [0, n - 1]
 *Status:* Highly tested
*/
template<class T, T m_(T, T)> struct segment_tree{
  int n; vector<T> ST;
  segment_tree(){}
  segment_tree(vector<T> &a){
    n = a.size(); ST.resize(n << 1);
    for (int i=n;i<(n<<1);i++)ST[i]=a[i-n];
    for (int i=n-1;i>0;i--)ST[i]=m_(ST[i<<1],ST[i<<1|1]);
  }
  void update(int pos, T val){ // replace with val
    ST[pos += n] = val;
    for (pos >>= 1; pos > 0; pos >>= 1)
      ST[pos] = m_(ST[pos<<1], ST[pos<<1|1]);
  }
  T query(int l, int r){ // [l, r]
    T ansL, ansR; bool hasL = 0, hasR = 0;
    for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
      if (l & 1)
        ansL=(hasL?m_(ansL,ST[l++]):ST[l++]),hasL=1;
      if (r & 1)
        ansR=(hasR?m_(ST[--r],ansR):ST[--r]),hasR=1;
    }
    if (!hasL) return ansR; if (!hasR) return ansL;
    return m_(ansL, ansR);
  }
};
```

10.7 segment tree lazy

```
/*
 *Description:* Segment tree but with range updates, build
$O(n)$, query and update $O(log n)$
 *Status:* Highly tested
*/
template<
```

```
  class T1, // answer value stored on nodes
  class T2, // lazy update value stored on nodes
  T1 merge(T1, T1),
  void pushUpd(T2& /*parent*/, T2& /*child*/, int, int, int,
int), // push update value from a node to another. parent ->
child
  void applyUpd(T2&, T1&, int, int)           // apply the
update value of a node to its answer value. upd -> ans
>
struct segment_tree_lazy{
  vector<T1> ST; vector<T2> lazy; vector<bool> upd;
  int n;
  void build(int i, int l, int r, vector<T1>&values){
    if (l == r){
      ST[i] = values[l];
      return;
    }
    build(i << 1, l, (l + r) >> 1, values);
    build(i << 1 | 1, (l + r) / 2 + 1, r, values);
    ST[i] = merge(ST[i << 1], ST[i << 1 | 1]);
  }
  segment_tree_lazy() {}
  segment_tree_lazy(vector<T1>&values){
    n = values.size(); ST.resize(n << 2 | 3);
    lazy.resize(n << 2 | 3); upd.resize(n << 2 | 3, false);
    build(1, 0, n - 1, values);
  }
  void push(int i, int l, int r){
    if (upd[i]){
      applyUpd(lazy[i], ST[i], l, r);
      if (l != r){
        pushUpd(lazy[i], lazy[i << 1], l, r, l, (l + r) /
2);
        pushUpd(lazy[i], lazy[i << 1 | 1], l, r, (l + r) / 2
+ 1, r);
        upd[i << 1] = 1;
        upd[i << 1 | 1] = 1;
      }
      upd[i] = false;
      lazy[i] = T2();
    }
  }
  void update(int i, int l, int r, int a, int b, T2 &u){
    if (l >= a and r <= b){
      pushUpd(u, lazy[i], a, b, l, r);
      upd[i] = true;
    }
    push(i, l, r);
    if (l > b or r < a) return;
    if (l >= a and r <= b) return;
    update(i << 1, l, (l + r) >> 1, a, b, u);
    update(i << 1 | 1, (l + r) / 2 + 1, r, a, b, u);
    ST[i] = merge(ST[i << 1], ST[i << 1 | 1]);
  }
  void update(int a, int b, T2 u){
    if (a > b){
      update(0, b, u);
      update(a, n - 1, u);
      return ;
    }
    update(1, 0, n - 1, a, b, u);
  }
  T1 query(int i, int l, int r, int a, int b){
    push(i, l, r);
    if (a <= l and r <= b)
      return ST[i];
    int mid = (l + r) >> 1;
    if (mid < a)
      return query(i << 1 | 1, mid + 1, r, a, b);
    if (mid >= b)
      return query(i << 1, l, mid, a, b);
    return merge(query(i << 1, l, mid, a, b), query(i << 1 |
1, mid + 1, r, a, b));
  }
  T1 query(int a, int b){
    if (a > b) return merge(query(a, n - 1), query(0, b));
    return query(1, 0, n - 1, a, b);
  }
};



// To use it, you must define:
//   1. merge(T1 a, T1 b) → combina dos nodos hijos en un
valor padre
//   2. pushUpd(T2 &parent, T2 &child, int l1, int r1, int
l2, int r2) →
//      propaga la actualización del nodo padre al hijo
//   3. applyUpd(T2 &upd, T1 &ans, int l, int r) → aplica el
valor lazy (T2)
//      al valor del nodo (T1)
//
/*
Cuando uses esta plantilla, necesitas proveer 3 funciones:
merge(T1 a, T1 b)
Define cómo combinar los valores de dos hijos.
Ejemplo: para un segmento que guarda sumas → return a + b;
pushUpd(T2 &parent, T2 &child, int Lp, int Rp, int Lc, int
Rc)
Define cómo se transmite una actualización pendiente de un
nodo a sus hijos.
Ejemplo: si las actualizaciones son aditivas → child +=
parent;
applyUpd(T2 &upd, T1 &node, int l, int r)
Define cómo aplicar una actualización a un nodo.
Ejemplo: para suma → node += (r - l + 1) * upd;
*/
//ejemplo suma acum

long long merge(long long a, long long b){ return a + b; }


void pushUpd(long long &p, long long &c, int Lp, int Rp, int
Lc, int Rc){
  c += p; // suma acumulativa
}

void applyUpd(long long &u, long long &node, int l, int r){
  node += (r - l + 1) * u;
}


//ej uso
vector<long long> v = {1, 2, 3, 4, 5};
segment_tree_lazy<long long, long long, merge, pushUpd,
applyUpd> st(v);

st.update(1, 3, 2LL);  // sumar 2 al rango [1, 3]
cout << st.query(0, 4); // suma total
```